



Computer Vision for Autonomous Vehicles

Lecture 5 and 6: Quadrotor Control

*Instructor: Golnaz Habibi, PhD
University of Oklahoma
Fall 2024*

August 29th, 2024



Announcement

- Lab 2 (ROS and TF) posted. Deadline, **Monday Sept 9th**.
- **Lab 2 all parts needs to be done individually.**
- Deadline for team assignment, Sept 9th.
- **No class next Thursday Sept 12 (Career Fair).**
- Update on the Docker:
 - Please make sure you pull rosdocker repository
 - Please make sure you pull new docker image

Today

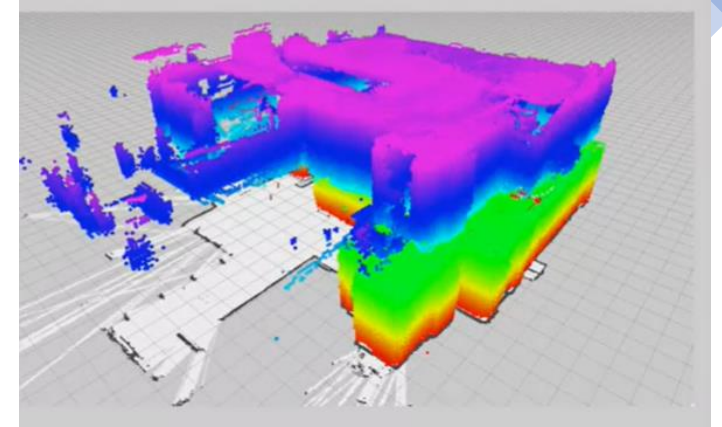
- Quadrotor Dynamic and Control
- Review Lab 2

Autonomous Navigation System

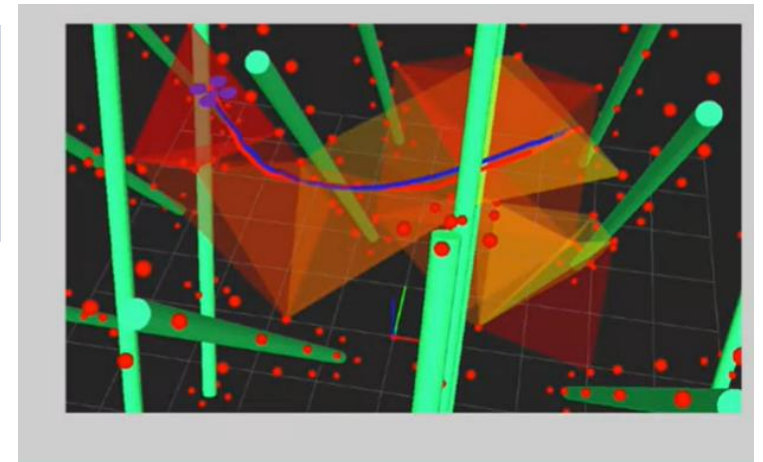
State Estimation

*estimate the position and velocity
(including rotation and angular
velocity of the robot)*

Mapping



Planning



Control

*command motors and produce
desired actions in order to navigate
to desired state*

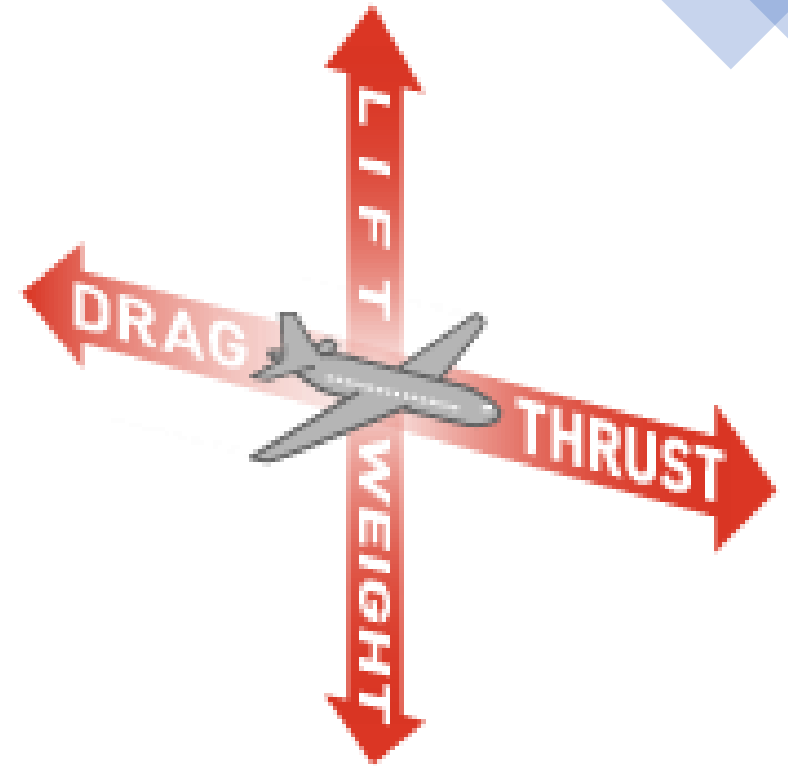
In this course...

- Quadrotors navigation in simulation
- Race car navigation (mainly final project)



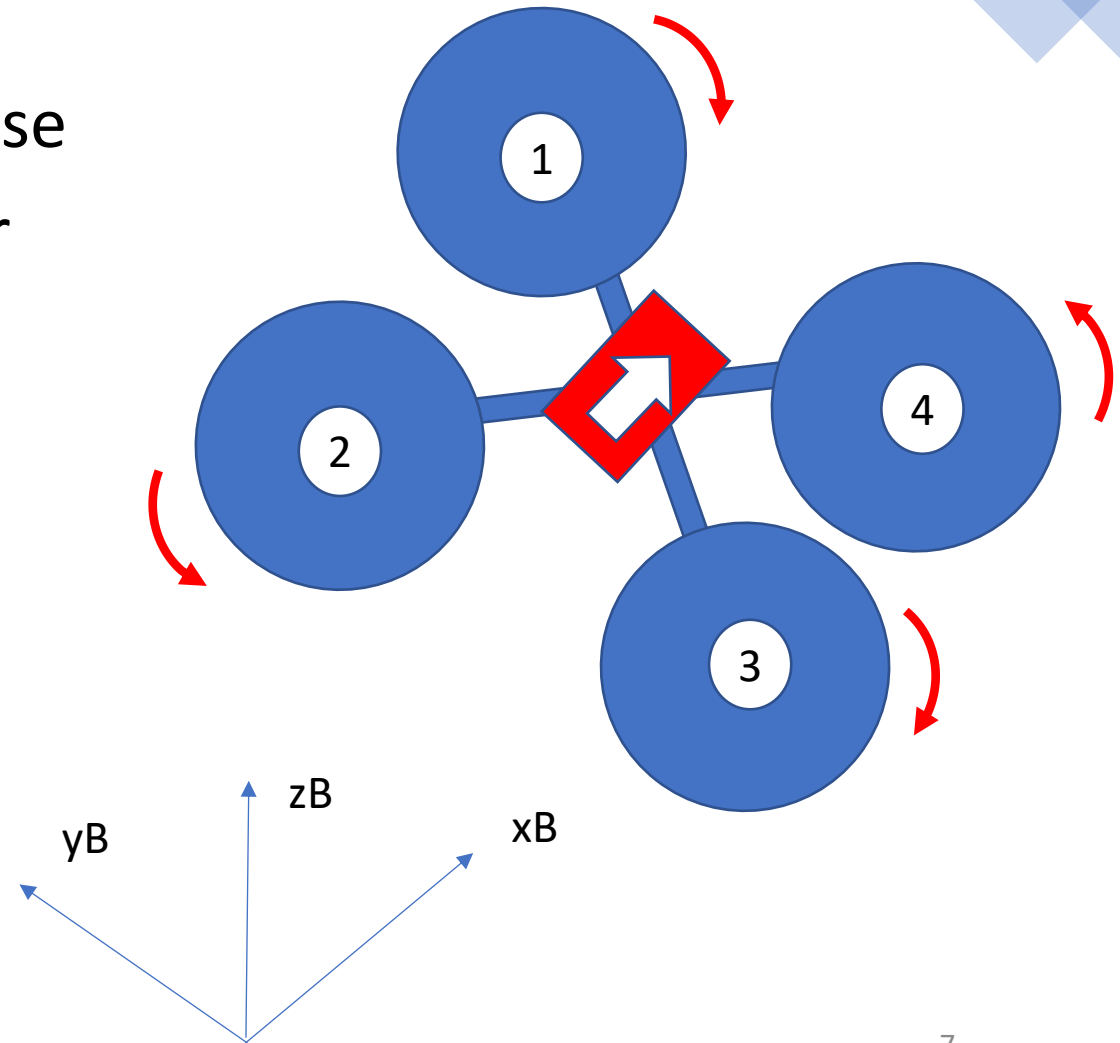
Aerial Navigation System

1. **Weight** : gravity
2. **Lift** is acted as the opposite of weight
3. **Thrust** forces the quadrotor to move to the desired direction
4. **Drag** is the force that acts opposite to the direction of motion.
Drag is caused by friction and differences in air pressure.



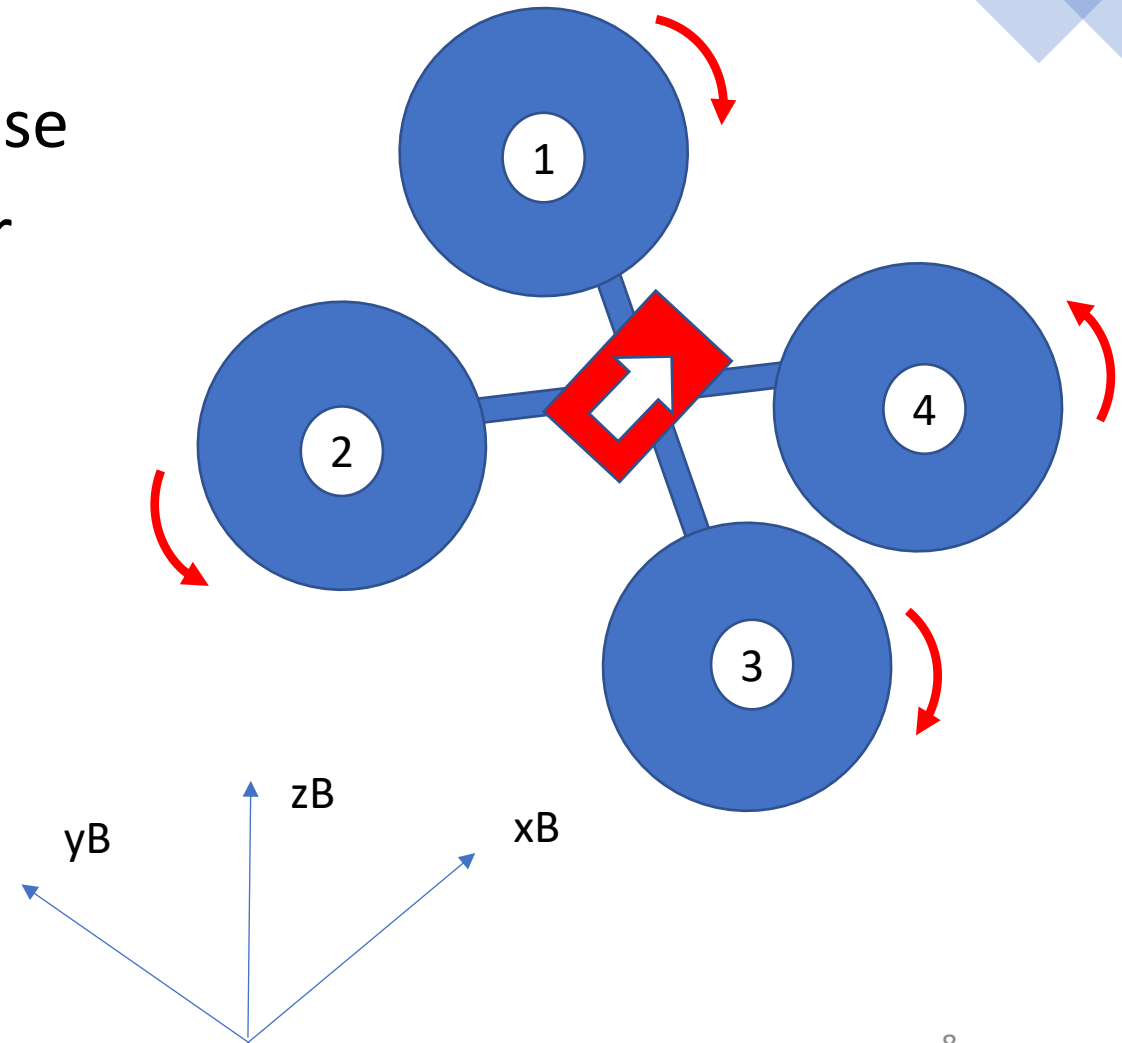
Quadrotor Dynamics

- Propellers 1 and 3 are rotating clockwise
- Propellers 2 and 4 are rotating counter clockwise
- A quadrotor changes its orientation, depending on the velocity of these propellers



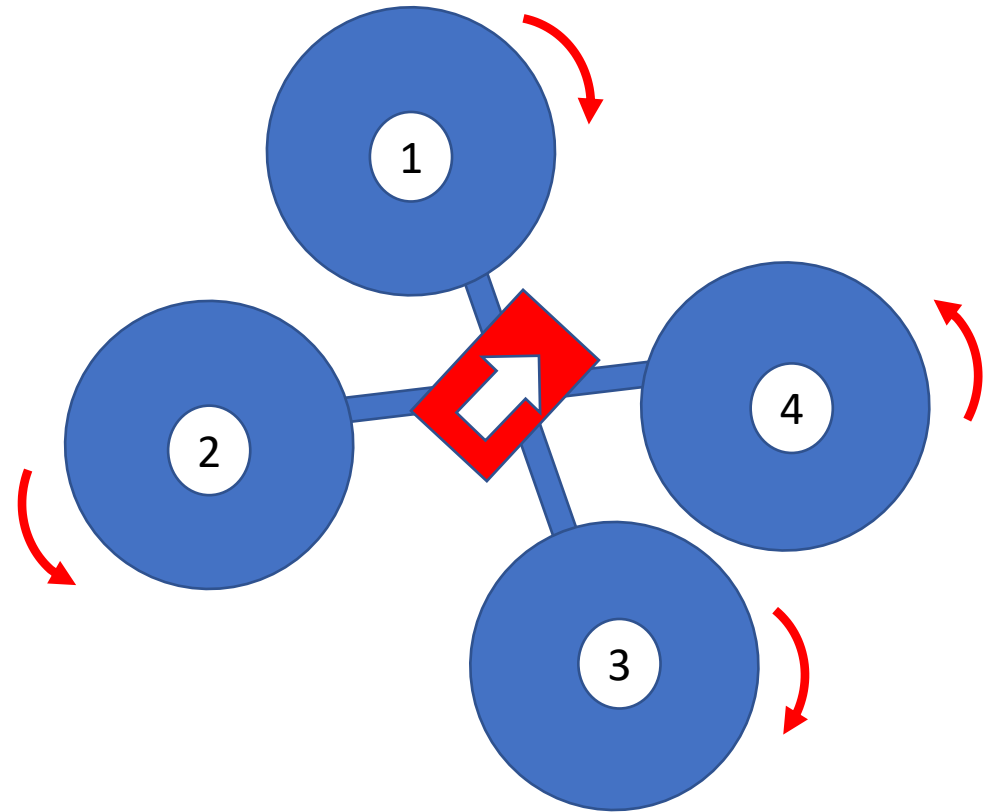
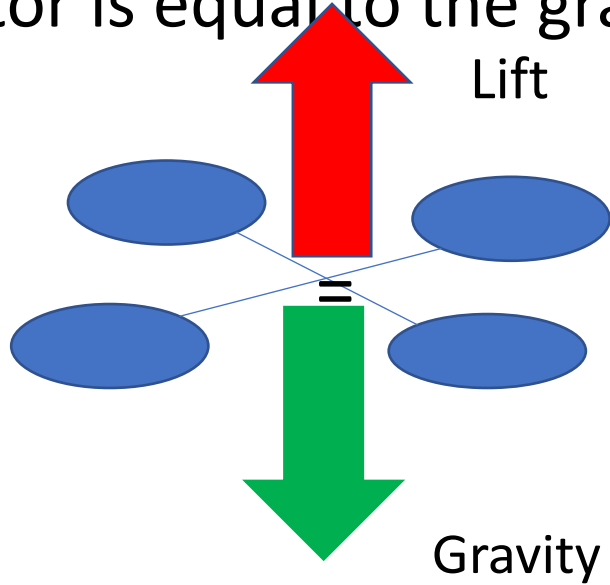
Quadrotor Dynamics

- Propellers 1 and 3 are rotating clockwise
- Propellers 2 and 4 are rotating counter clockwise
- A quadrotor changes its orientation, depending on the velocity of these propellers

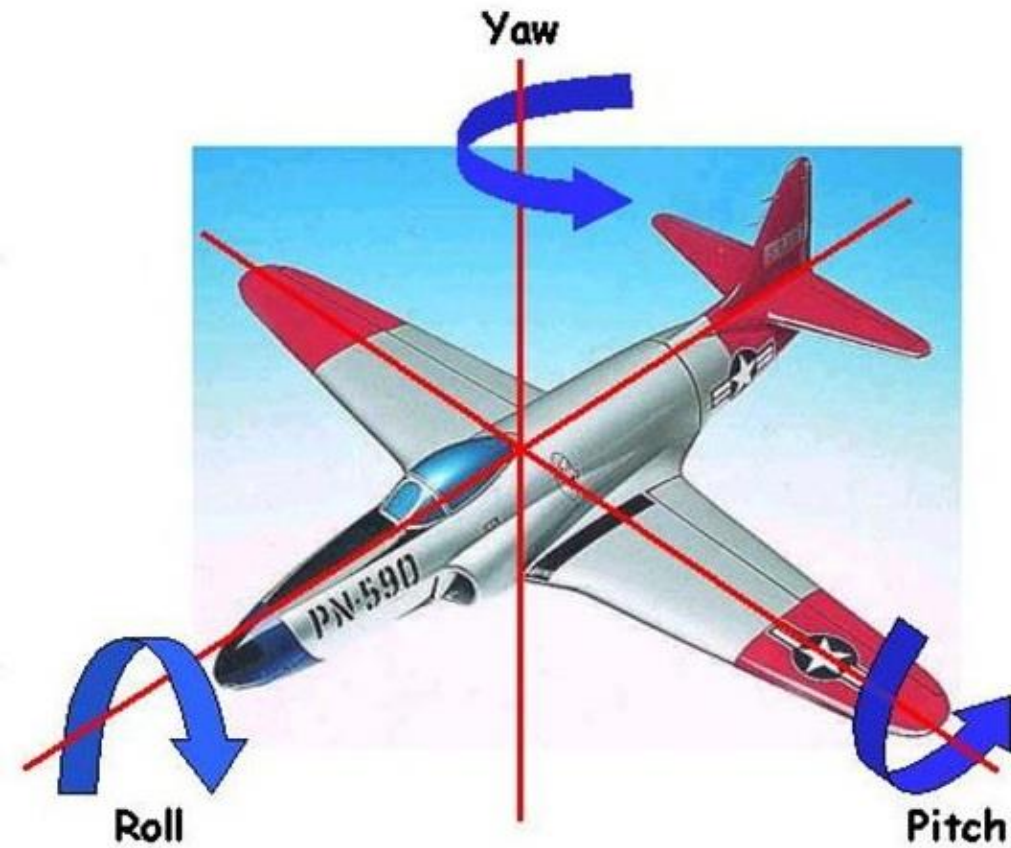
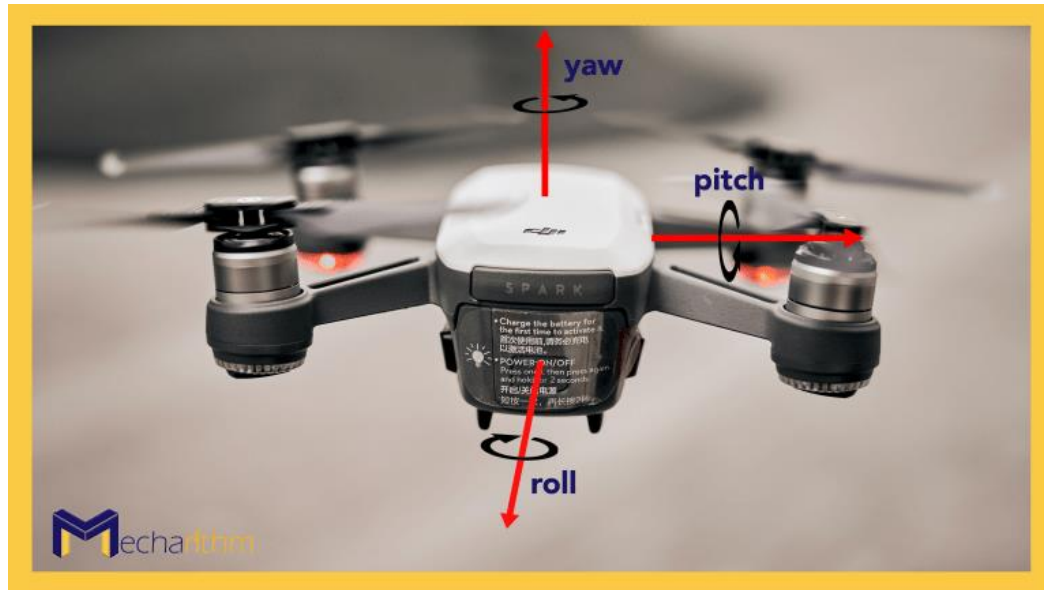


Hovering

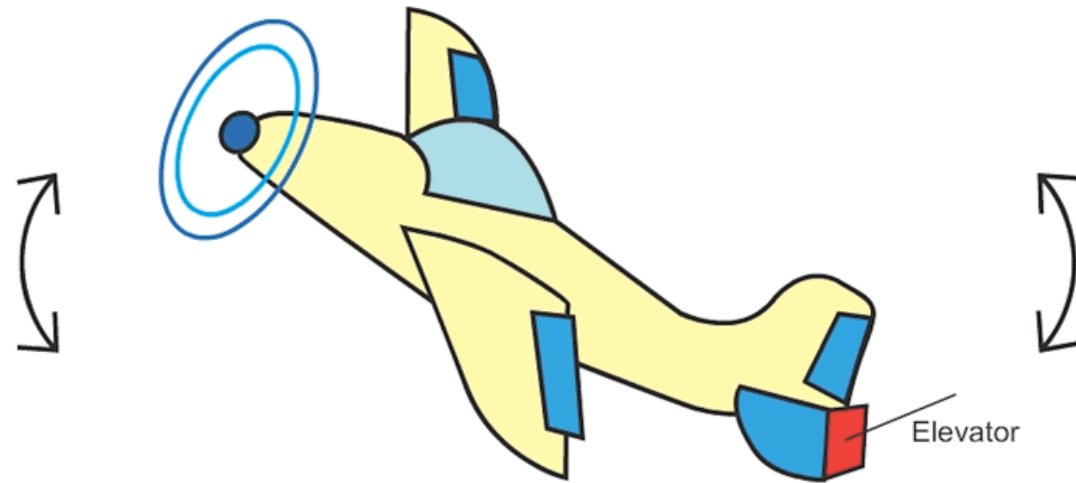
- When the lift force is equal to gravity: Here the total thrust forces in quadrotor is equal to the gravity



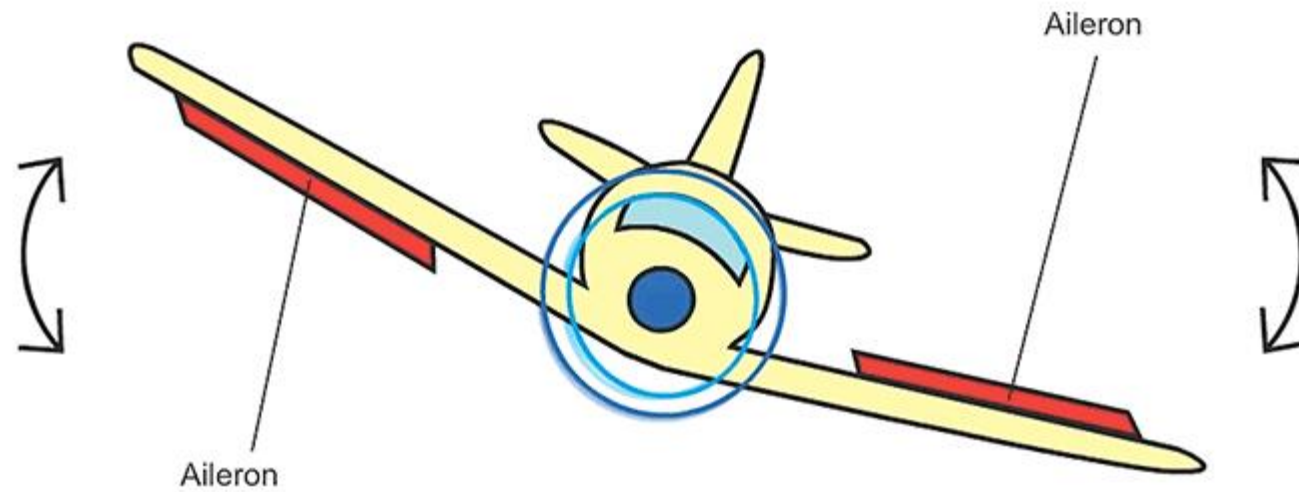
Pitch/Roll/Yaw



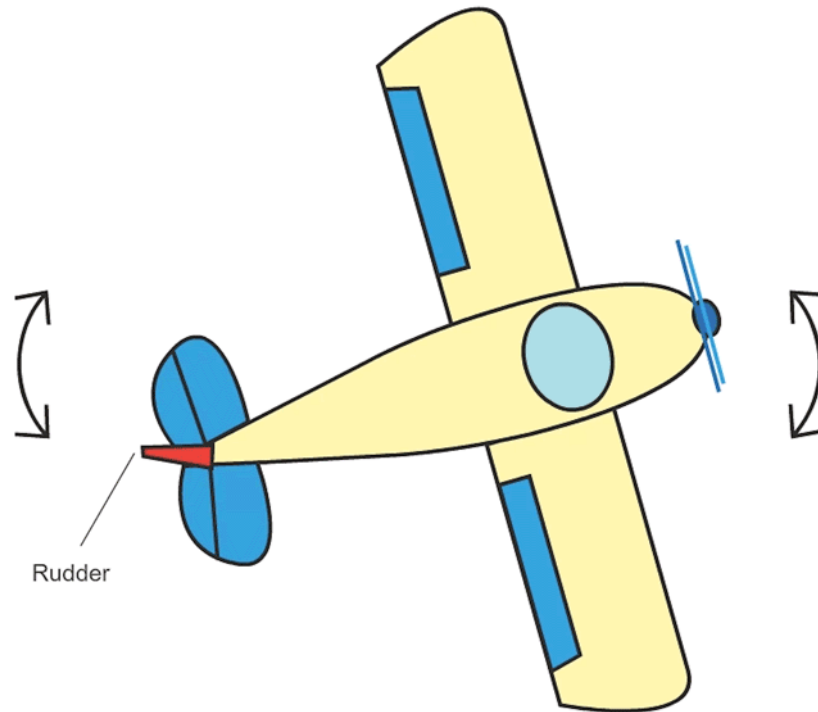
Pitch!



Roll!

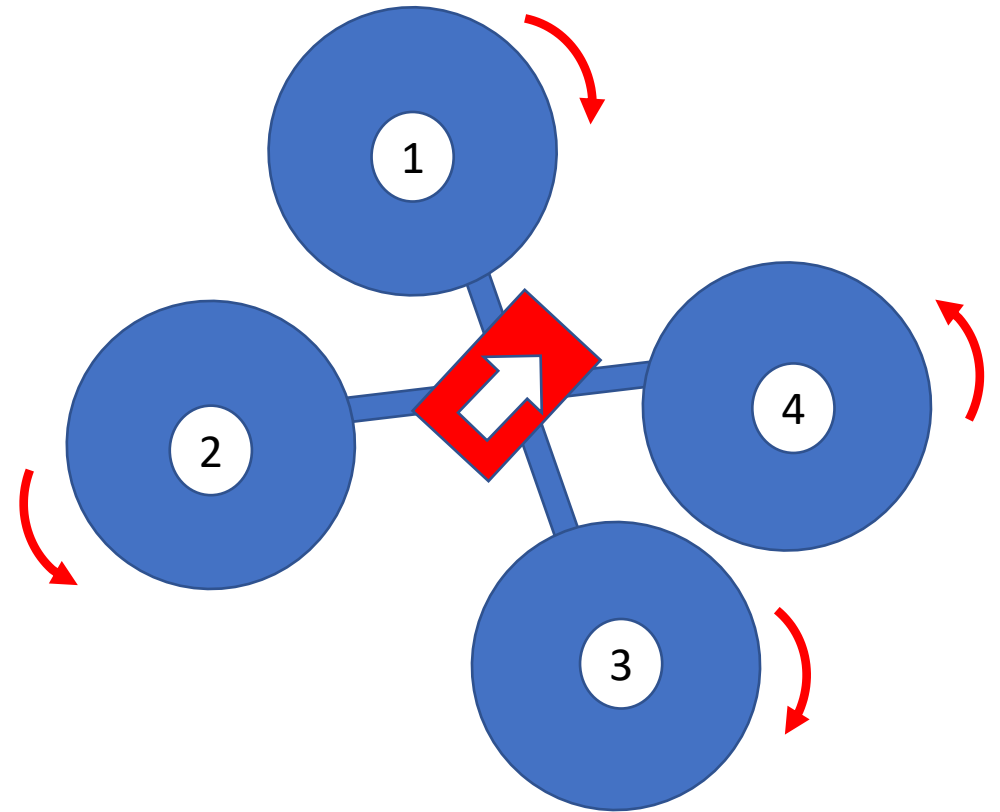
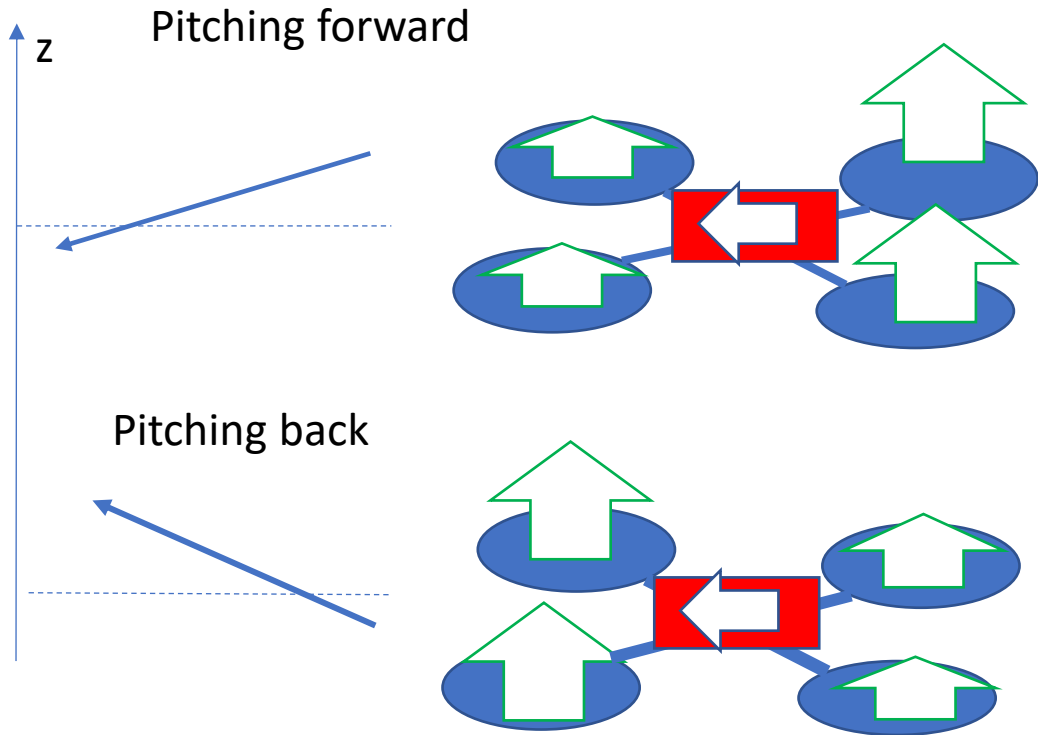


Yaw!



Pitching

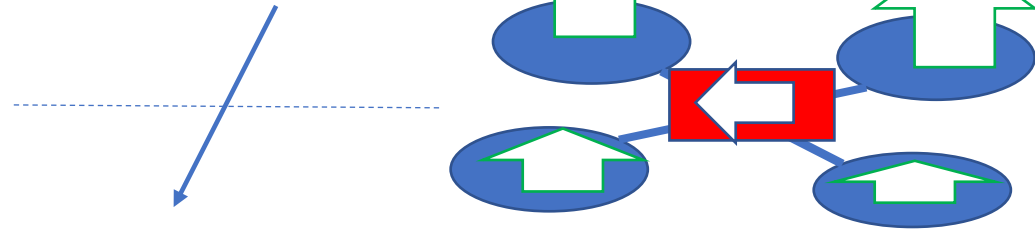
- Motors on the back are rotating faster (top) or slower (bottom) than the motors on the front.



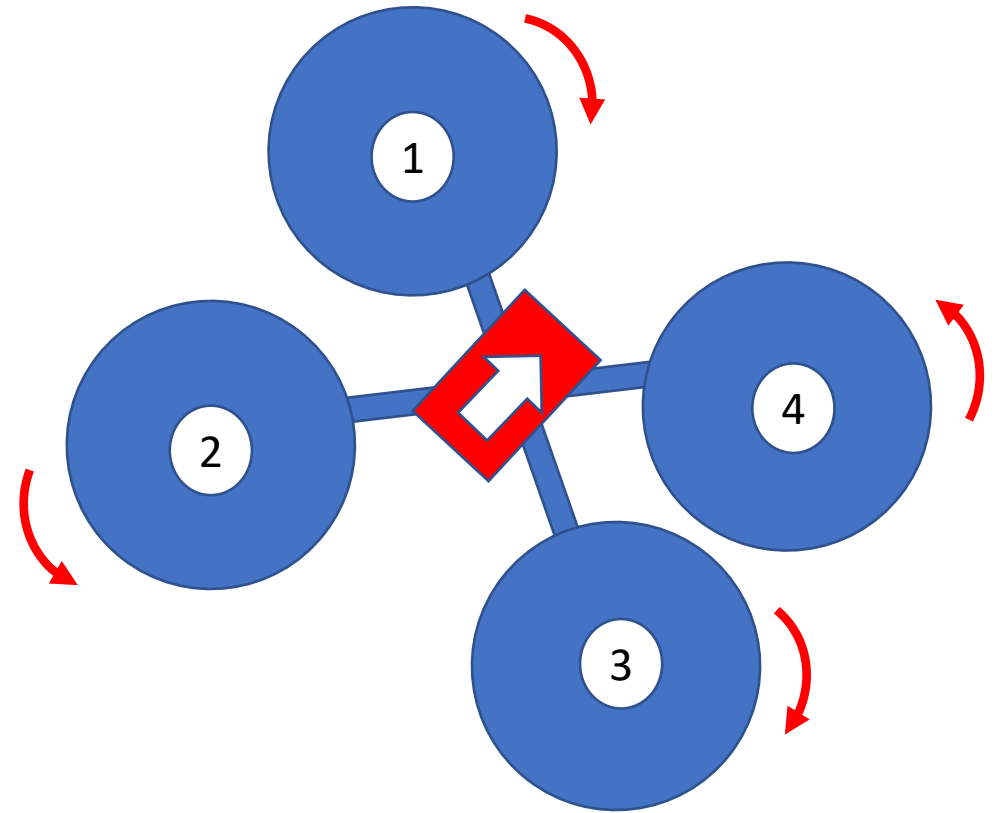
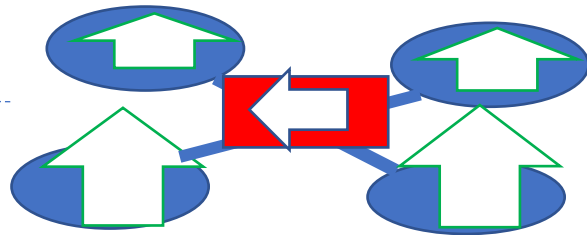
Rolling

- Motors on the left are rotating faster (top) or slower than motors on the right

Rolling left

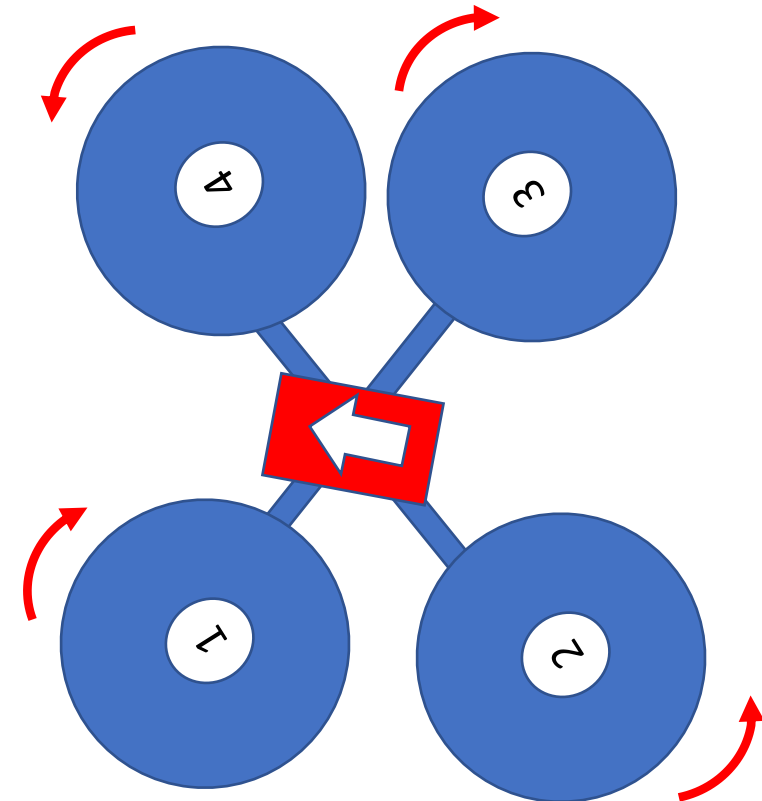
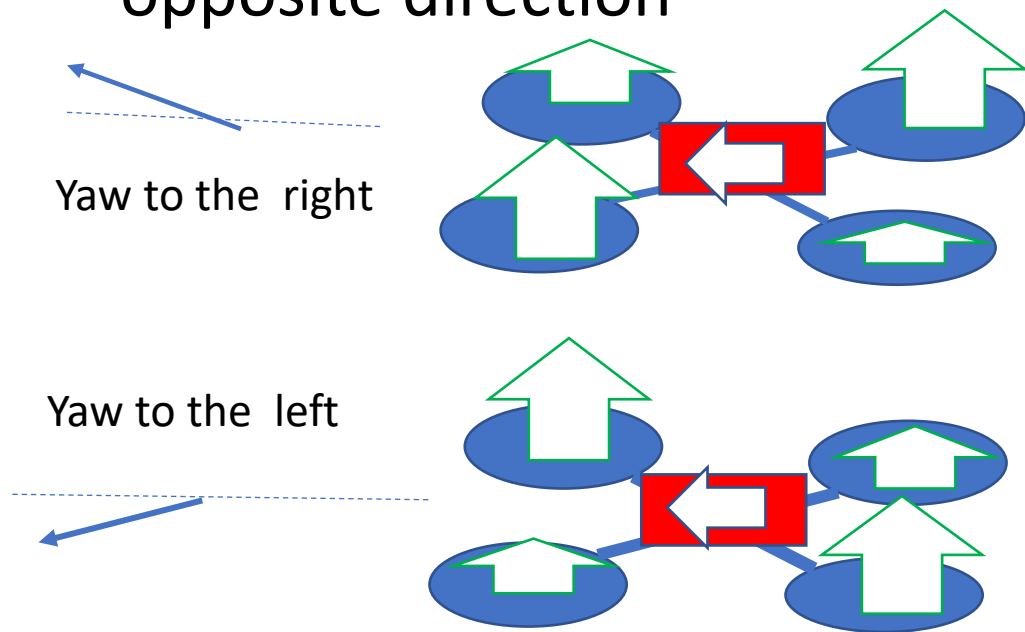


Rolling right

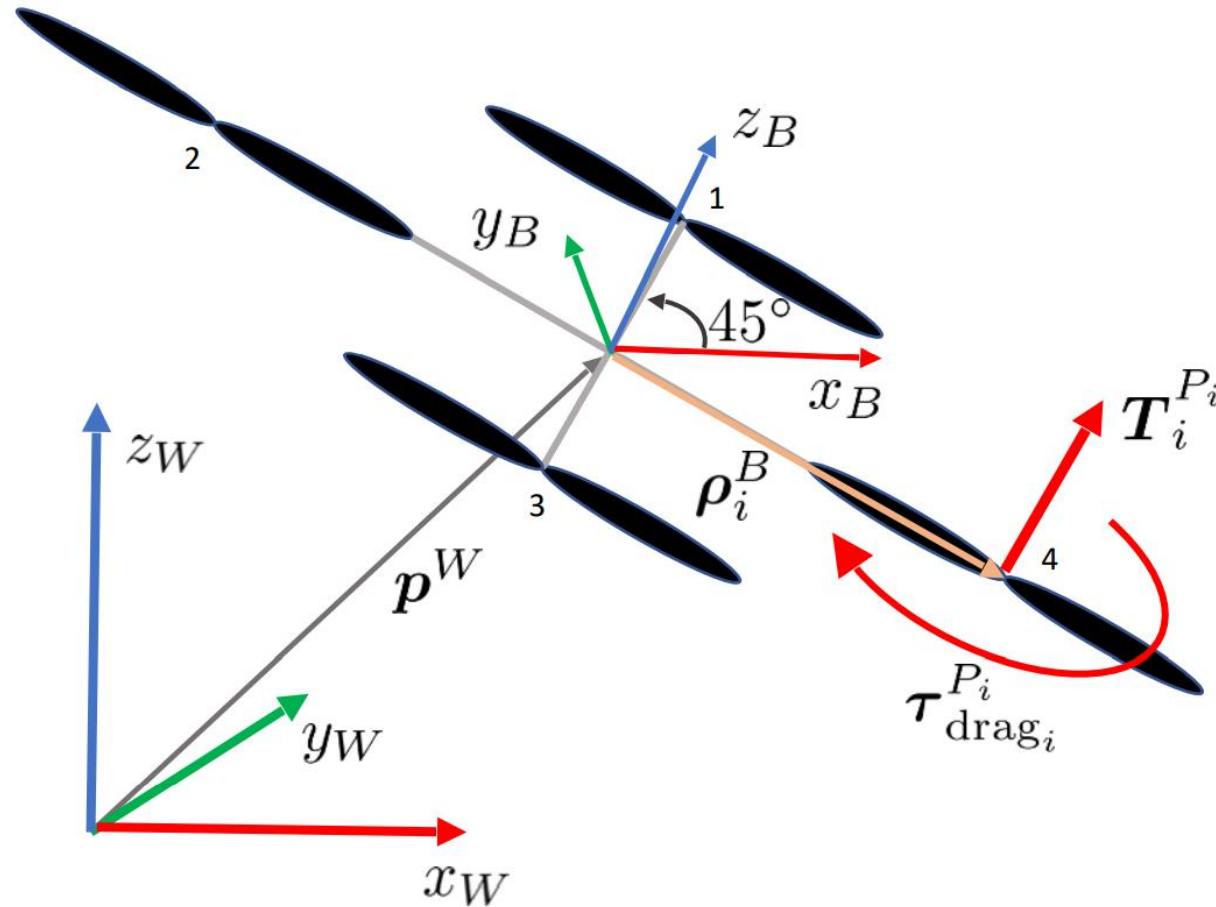


Yaw

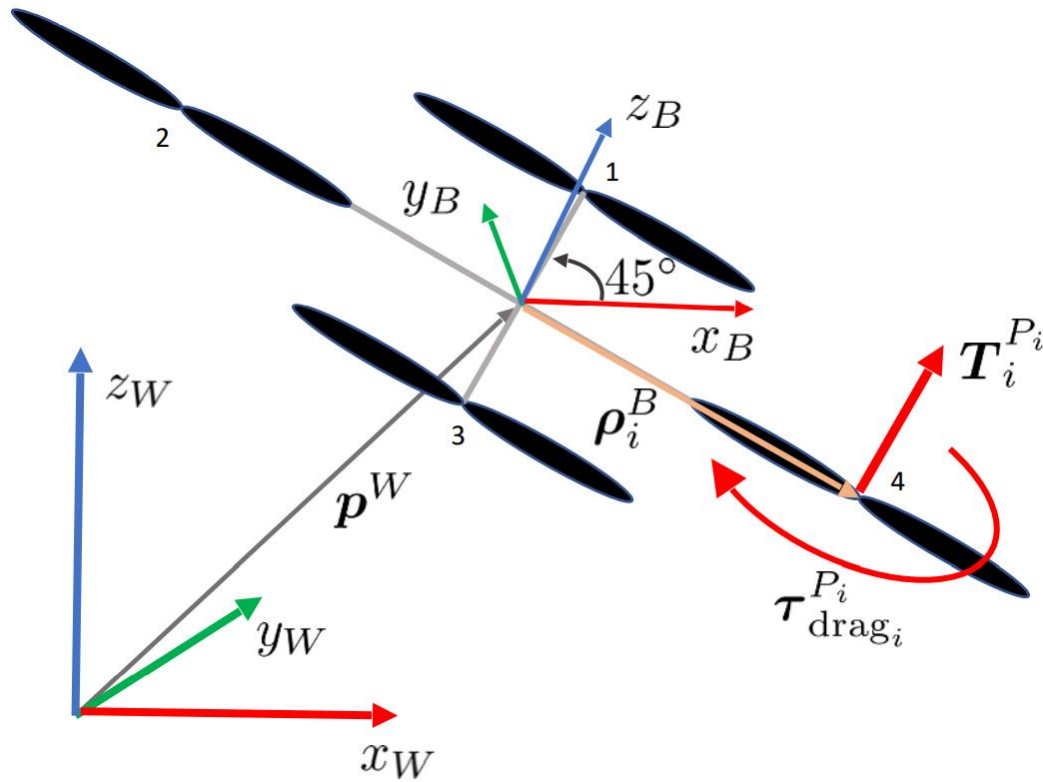
- Motors that rotating counter clockwise are rotating faster (top- yaw to the right) or slower (bottom- yaw to the right) than the motors that rotating clockwise
- Note that based on the 3rd Newton's law, the drone is rotating in the opposite direction



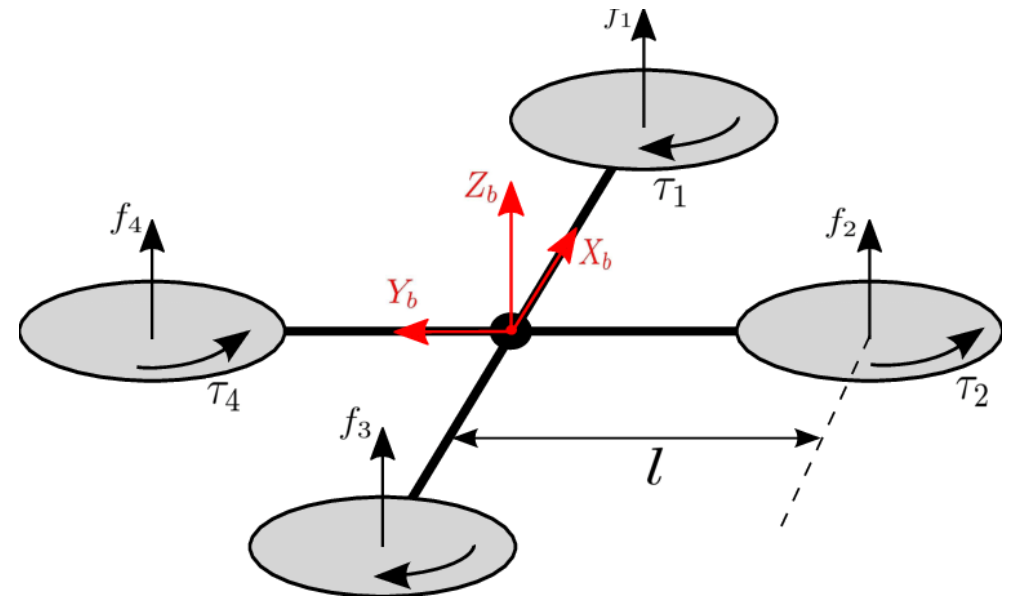
Quadrotor Dynamics



Body frames –different settings



This class setting



Another setting

Quadrotor Model

Let us introduce the Newton-Euler equation, describing the translational and rotational dynamics of a rigid body:

$$\begin{bmatrix} \mathbf{f}^{\text{w}} \\ \boldsymbol{\tau}^{\text{B}} \end{bmatrix} = \begin{bmatrix} m\mathbf{I}_3 & \mathbf{0} \\ \mathbf{0} & \mathcal{J} \end{bmatrix} \begin{bmatrix} \mathbf{a}^{\text{w}} \\ \boldsymbol{\alpha}^{\text{B}} \end{bmatrix} + \begin{bmatrix} \mathbf{0} \\ \boldsymbol{\omega}^{\text{B}} \times \mathcal{J}\boldsymbol{\omega}^{\text{B}} \end{bmatrix} \quad (6.1)$$

with symbols as defined in Table 6.1. Note that the translational part is the usual “ $\mathbf{f} = m\mathbf{a}$ ” relation, while the rotational part is slightly more complicated. In particular, the extra cross product term at the end emerges when expressing the torque in the body frame (see [https://en.wikipedia.org/wiki/Euler%27s_equations_\(rigid_body_dynamics\)](https://en.wikipedia.org/wiki/Euler%27s_equations_(rigid_body_dynamics)) for a justification).

$\mathbf{f}^{\text{w}} \in \mathbb{R}^3$	applied total forces expressed in the world frame
$\boldsymbol{\tau}^{\text{B}} \in \mathbb{R}^3$	applied total torques expressed in the body frame
$m \in \mathbb{R}_+$	body mass
$\mathcal{J} \in \mathbb{R}^{3 \times 3}$	moment of inertia about the center of mass
$\mathbf{I}_n$	$n \times n$ identity matrix
$\mathbf{a}^{\text{w}} \in \mathbb{R}^3$	translational acceleration of the center of mass expressed in the world frame
$\boldsymbol{\alpha}^{\text{B}} \in \mathbb{R}^3$	angular acceleration of the body expressed in the body frame
$\boldsymbol{\omega}^{\text{B}} \in \mathbb{R}^3$	angular velocity of the body expressed in the body frame

Considering Gravity...

In the following, we discuss the nature of the forces $\mathbf{f}^w$ and the torques $\boldsymbol{\tau}^B$ when the rigid body is a quadrotor. The first external force we consider is *gravity*, which we make explicit in our model and move to the right-hand-side of (6.1) (note: $\mathbf{g}^w \in \mathbb{R}^3$ is the gravity vector expressed in the world frame):

$$\begin{bmatrix} \mathbf{f}^w \\ \boldsymbol{\tau}^B \end{bmatrix} = \begin{bmatrix} m\mathbf{I}_3 & \mathbf{0} \\ \mathbf{0} & \mathcal{J} \end{bmatrix} \begin{bmatrix} \mathbf{a}^w \\ \boldsymbol{\alpha}^B \end{bmatrix} + \begin{bmatrix} -m\mathbf{g}^w \\ \boldsymbol{\omega}^B \times \mathcal{J}\boldsymbol{\omega}^B \end{bmatrix} \quad (6.2)$$

Other forces and torques are due to aerodynamic effects. The two main aerodynamic effects for a quadrotor are:

- Thrust Force
- Rotor Drag

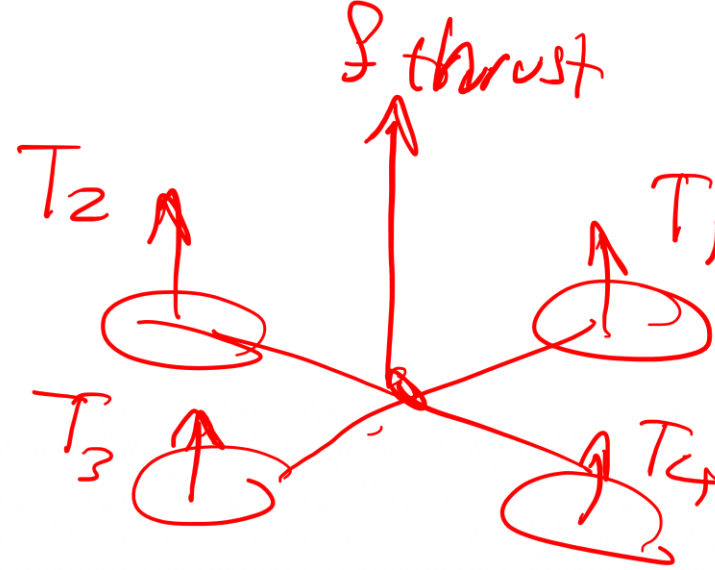
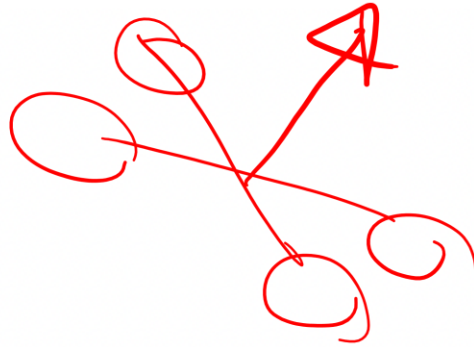
Other forces...

- Hub Force
- Hub Moment
- Rolling Moment
- Ground effect

At low speed ($< 10\text{m/s}$), the latter are by more than one order of magnitude smaller than the rotor drag and the thrust force, and can therefore be neglected. A more detailed explanation of these effects is given in [\[1\]](#).

Forces and Torques..

$$\mathbf{f}_{thrust}^B = \sum_{i=1}^4 \mathbf{R}_{P_i}^B \mathbf{T}_i^{P_i},$$



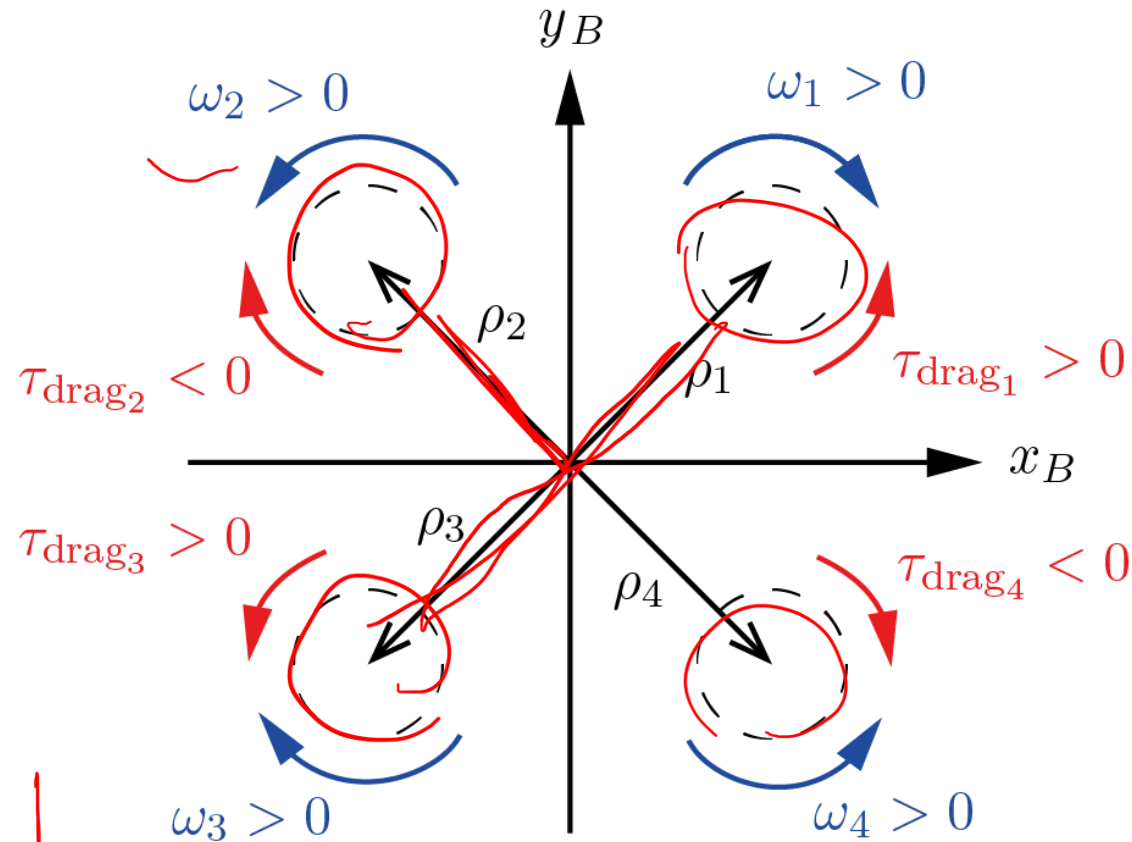
(6.3)

with $\mathbf{R}_{P_i}^B$ being the rotation matrix from propeller- to body-frame ($\mathbf{R}_{P_i}^B = \mathbf{I}_3$ for standard quadrotors).

$$\mathbf{T}_i^{P_i} = c_f \omega_i |\omega_i| \mathbf{e}_3,$$

$$\omega_i \Rightarrow (-1)^{i+1} |\omega_i|^2 = (-1)^{i+1} \omega_i^2$$

This course (TESSE simulator) convention



$$\begin{pmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_4 \end{pmatrix} \rightarrow \begin{pmatrix} a \\ \alpha \end{pmatrix}$$

(top view of the quadrotor, $\|\rho_i\|_2 = d$)

Forces and Torques..

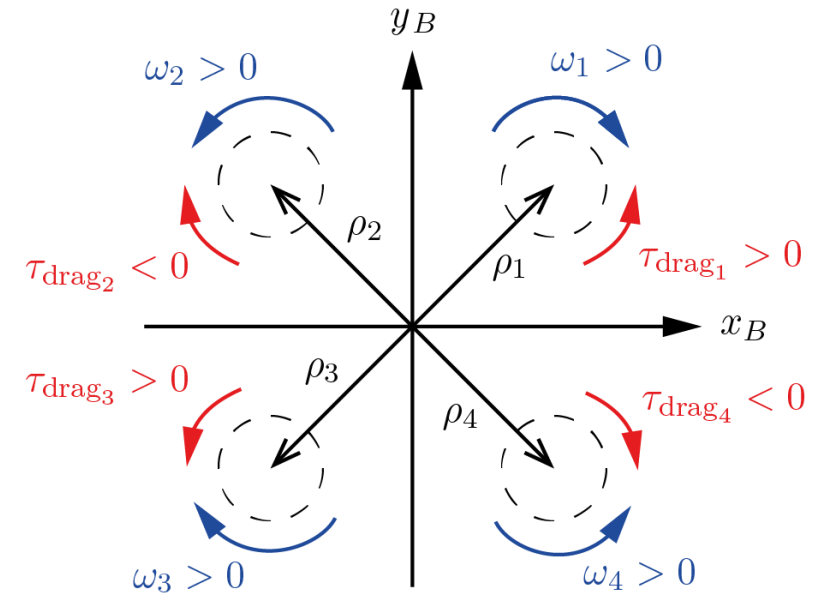
$$\tau^B = \tau_{drag}^B + \tau_{thrust}^B$$

$$\tau_{thrust}^B = \sum_{i=1}^4 (\rho_i^B \times R_{P_i}^B T_i^{P_i}),$$

$$\tau_{drag}^B = \sum_{i=1}^4 R_{P_i}^B \tau_{drag_i}^{P_i}$$

$$\tau_{drag_i}^{P_i} = (-1)^{(i+1)} c_d w_i |w_i| e_3$$

$$\tau = (-1)^{i+1} c_d w_i |w_i| e_3$$



(top view of the quadrotor, $\|\rho_i\|_2 = d$)

Let's recap...

$$\begin{bmatrix} \mathbf{f}^w \\ \boldsymbol{\tau}^B \end{bmatrix} = \begin{bmatrix} m\mathbf{I}_3 & \mathbf{0} \\ \mathbf{0} & \mathcal{J} \end{bmatrix} \begin{bmatrix} \mathbf{a}^w \\ \boldsymbol{\alpha}^B \end{bmatrix} + \begin{bmatrix} -m\mathbf{g}^w \\ \boldsymbol{\omega}^B \times \mathcal{J}\boldsymbol{\omega}^B \end{bmatrix} \quad (6.2)$$

~~$\begin{bmatrix} m\mathbf{a}^w \\ \mathcal{J}\boldsymbol{\alpha}^B \end{bmatrix}$~~

- Substitute with torque and Force:

$$\begin{bmatrix} m\mathbf{a}^w \\ \mathcal{J}\boldsymbol{\alpha}^B \end{bmatrix} = \begin{bmatrix} m\mathbf{g}^w \\ -\boldsymbol{\omega}^B \times \mathcal{J}\boldsymbol{\omega}^B \end{bmatrix} + \begin{bmatrix} \mathbf{R}_B^w & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_3 \end{bmatrix} \begin{bmatrix} \mathbf{f}_{thrust}^B \\ \boldsymbol{\tau}_{drag}^B + \boldsymbol{\tau}_{thrust}^B \end{bmatrix} = \begin{bmatrix} -m\mathbf{g}e_3 \\ -\boldsymbol{\omega}^B \times \mathcal{J}\boldsymbol{\omega}^B \end{bmatrix} + \begin{bmatrix} \mathbf{R}_B^w & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_3 \end{bmatrix} \mathbf{F}^w \quad (6.7)$$

Finally...

$$\begin{bmatrix} m\mathbf{a}^w \\ \mathcal{J}\boldsymbol{\alpha}^B \end{bmatrix} = \begin{bmatrix} m\mathbf{g}^w \\ -\boldsymbol{\omega}^B \times \mathcal{J}\boldsymbol{\omega}^B \end{bmatrix} + \begin{bmatrix} \mathbf{R}_B^w & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_3 \end{bmatrix} \begin{bmatrix} \mathbf{f}_{thrust}^B \\ \boldsymbol{\tau}_{drag}^B + \boldsymbol{\tau}_{thrust}^B \end{bmatrix} = \begin{bmatrix} -mg\mathbf{e}_3 \\ -\boldsymbol{\omega}^B \times \mathcal{J}\boldsymbol{\omega}^B \end{bmatrix} + \begin{bmatrix} \mathbf{R}_B^w & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_3 \end{bmatrix} \mathbf{F}\mathbf{w} \quad (6.7)$$

where we defined $\mathbf{w} = [w_1|w_1|, w_2|w_2|, w_3|w_3|, w_4|w_4|]^T$, we defined a suitable matrix $\mathbf{F}$, and we observed that for a typical choice of world frame (with the z axis pointing upwards) it holds $\mathbf{g}^w = -g\mathbf{e}_3$. We remark that the vector $\mathbf{F}\mathbf{w}$ essentially includes the forces (except gravity) and the torques in the body frame, which using (6.3)-(6.6) and assuming $\mathbf{R}_{P_i}^B = \mathbf{I}_3$, can be written as:

$$\left\{ \begin{bmatrix} f_x^B \\ f_y^B \\ f_z^B \\ \tau_x^B \\ \tau_y^B \\ \tau_z^B \end{bmatrix} \right\} = \mathbf{F}\mathbf{w} = \begin{bmatrix} c_f\mathbf{e}_3 & c_f\mathbf{e}_3 & c_f\mathbf{e}_3 & c_f\mathbf{e}_3 \\ c_d\mathbf{e}_3 + c_f\boldsymbol{\rho}_1^B \times \mathbf{e}_3 & -c_d\mathbf{e}_3 + c_f\boldsymbol{\rho}_2^B \times \mathbf{e}_3 & c_d\mathbf{e}_3 + c_f\boldsymbol{\rho}_3^B \times \mathbf{e}_3 & -c_d\mathbf{e}_3 + c_f\boldsymbol{\rho}_4^B \times \mathbf{e}_3 \end{bmatrix} \mathbf{w} \quad (6.8)$$

$\mathbf{w} = \begin{bmatrix} \omega_1 |\omega_1| \\ \omega_2 |\omega_2| \\ \omega_3 |\omega_3| \\ \omega_4 |\omega_4| \end{bmatrix}$

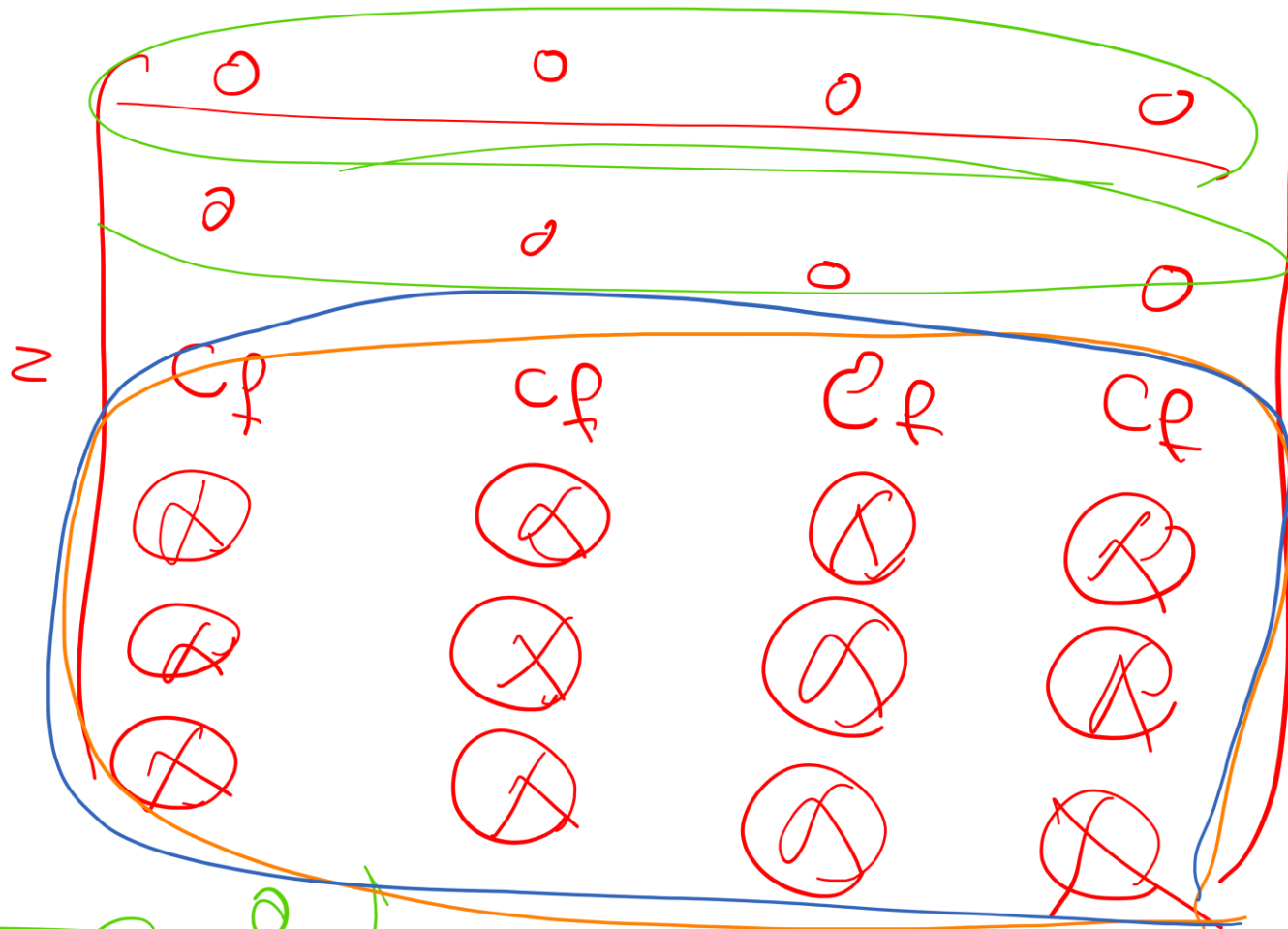
$$\begin{pmatrix} p_x \\ p_y \\ p_z \\ z_x \\ z_y \\ z_z \end{pmatrix} = \begin{pmatrix} c_F \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} & c_F \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} & c_F \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} & c_F \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \end{pmatrix} \begin{pmatrix} \omega_1/\omega_1 \\ \omega_2/\omega_2 \\ \omega_3/\omega_3 \\ \omega_4/\omega_4 \end{pmatrix} + c_F \underline{p_i^B \times e_3}$$

$$\begin{pmatrix} 0 \\ 0 \\ p_x \\ z_x \\ z_y \\ z_z \end{pmatrix}$$

$$F_z = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ \bigcirc & \bigcirc & \bigcirc & \bigcirc \end{pmatrix} \} \text{Constant}$$

6×4 4×1

f_n
 f_y
 f_z
 z_x
 z_y
 z_z



$\omega_1 |\omega_1|$
 $\omega_2 |\omega_2|$
 $\omega_3 |\omega_3|$
 $\omega_4 |\omega_4|$

f_x
 f_y
 f_z

z

$\omega_1 \omega_2 \omega_3 \omega_4$

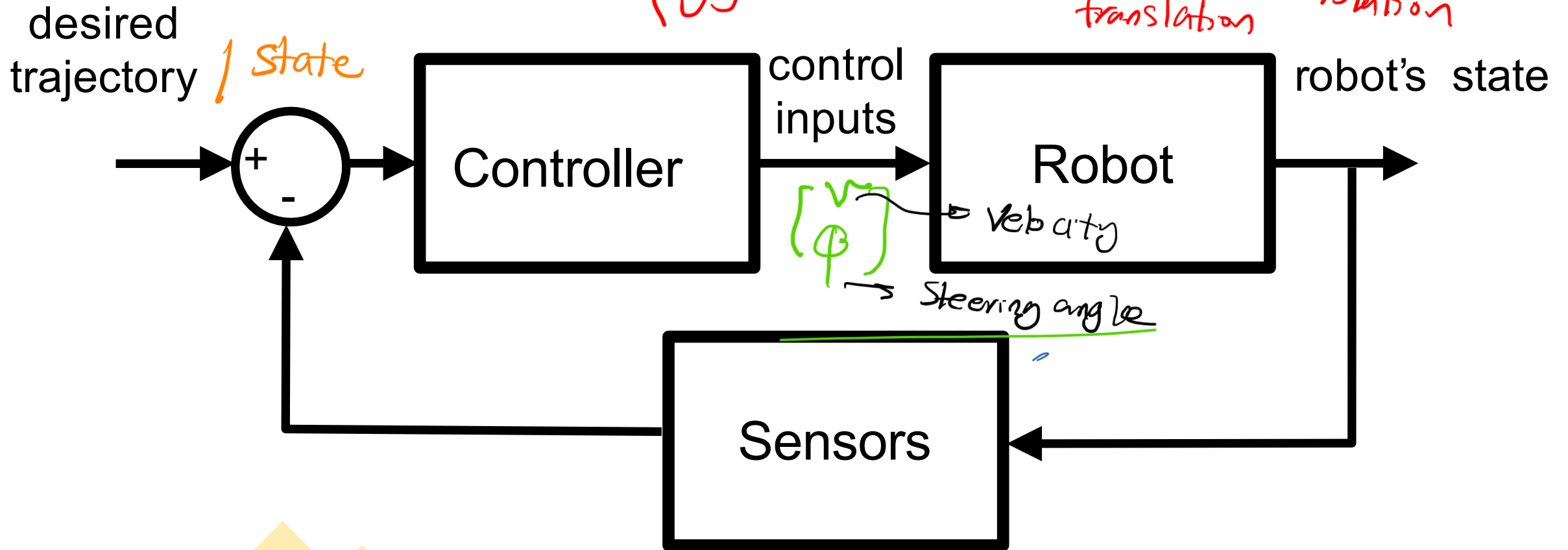
$\Rightarrow f_n = 0$ $f_y = 0$

IT

Announcement

- Lab 2 is due tonight.
- Please make sure you follow the instruction in the canvas carefully before working on the assignment.
- If you use docker (specially in the lab) and you need to get access to your code:
 - Make sure you do not put your code in docker container, it will be lost if you delete your container
 - Put in the home directory, but please try to get access inside the docker terminal
- Lab 3 will be out today. Deadline is is out, due time is Friday September 20th, 11:59 PM. Please start early!
- We plan to release the robots next week.
- Please make sure you only reach out to me if you have any questions on the assignment:
 - Please do not directly contact IT administrator. He is not TA of this class!

Closed loop controller



Quadrotor Control

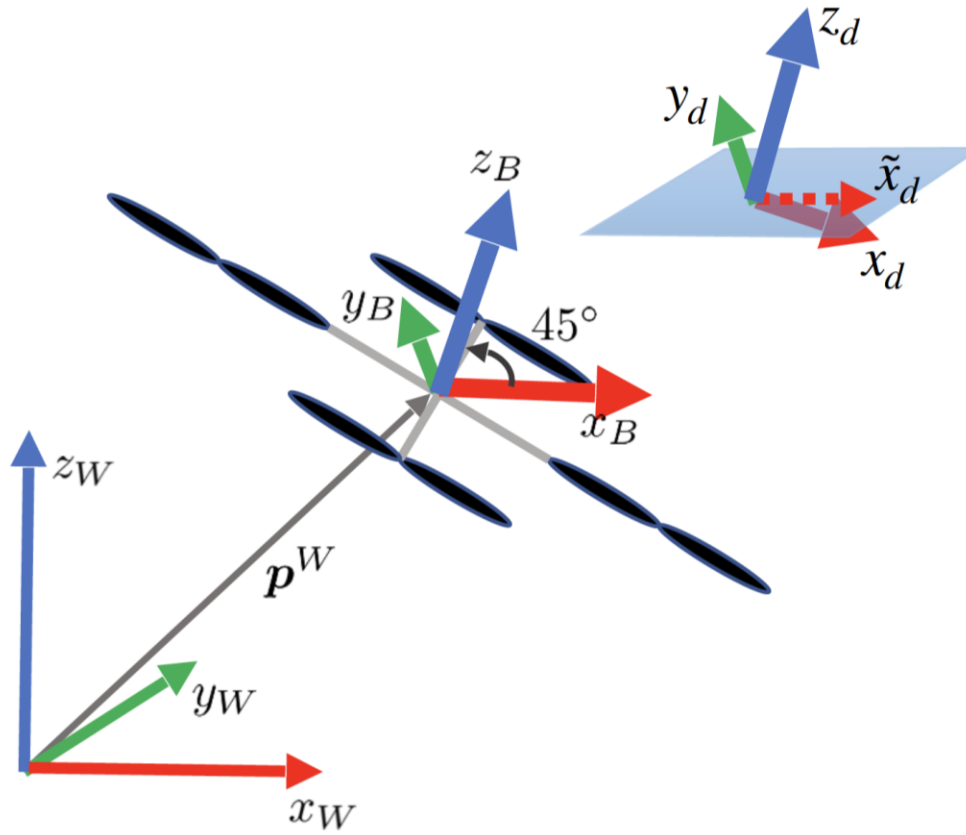
$$\left\{ \begin{array}{l} P \xrightarrow{w} P_d \\ R_B^w \xrightarrow{w} R_d^w \end{array} \right.$$

$$R_B^W = [x_B^W \ y_B^W \ z_B^W]$$

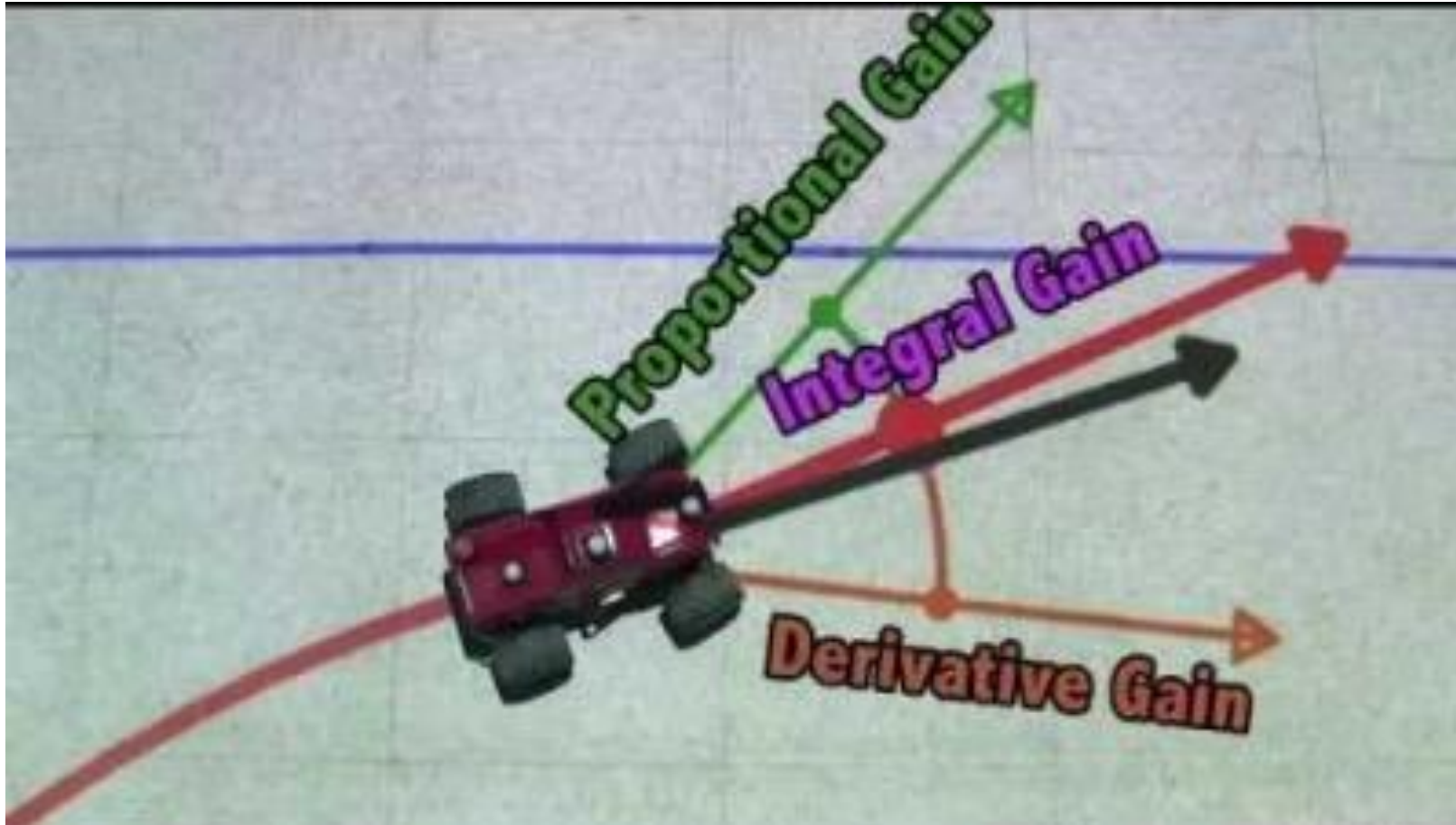
$$R_d^W = [x_d^W \ y_d^W \ z_d^W]$$

$$P_d^w = \begin{pmatrix} p_{x_d} & p_{y_d} & p_{z_d} \end{pmatrix}$$

desired position of drone



Videos: PID Control in Autonomous Driving

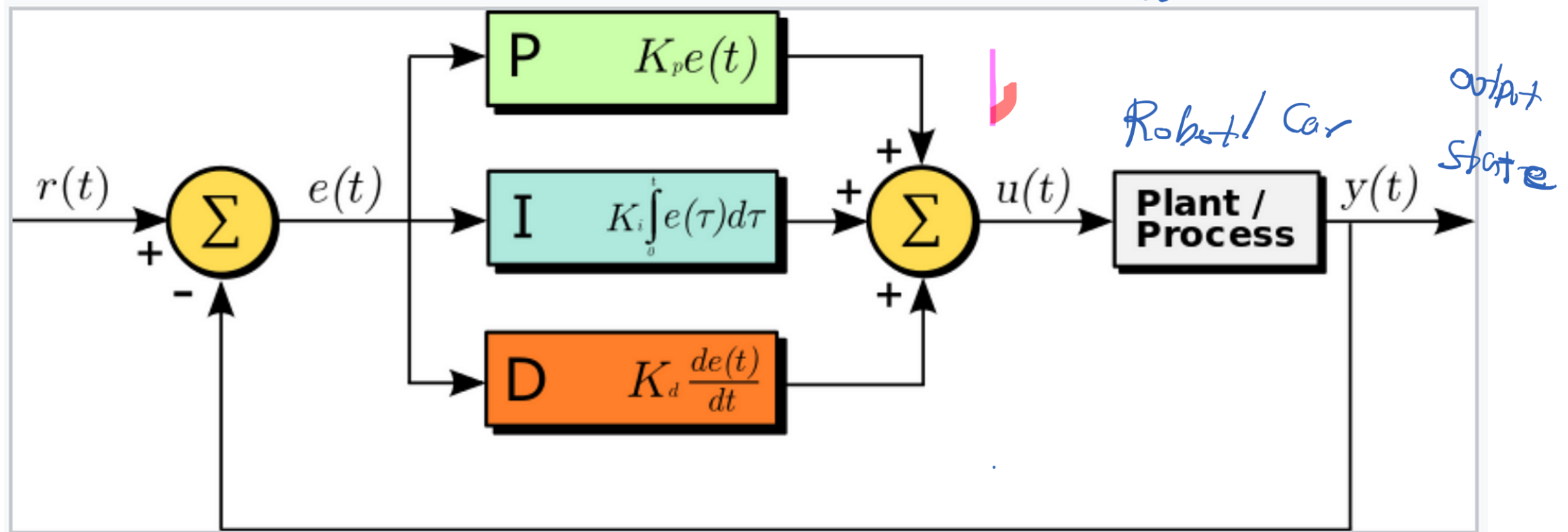


$$P : u(t) = k_p e(t)$$

PID controller

$$PI : u(t) = k_p e(t) + k_i \int_0^t e(\tau) d\tau$$

$$e(t) = r(t) - y(t) \quad PD : u(t) = k_p e(t) + k_d \frac{de(t)}{dt}$$

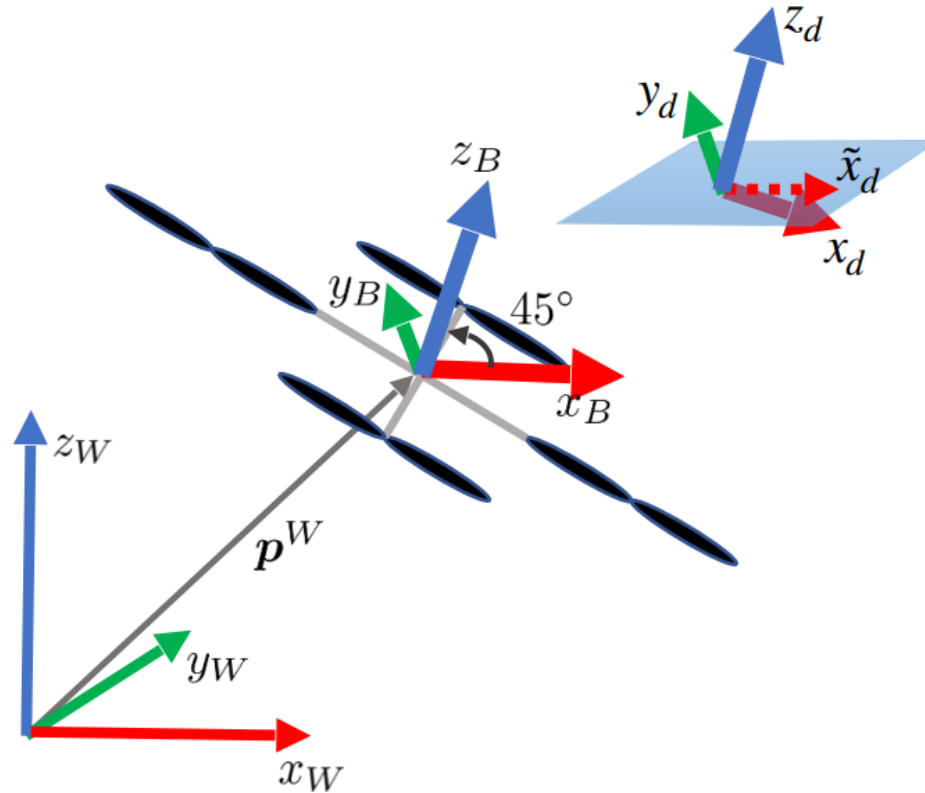


$$PID : u(t) = k_p e(t) + k_i \int_0^t e(\tau) d\tau + k_d \frac{de(t)}{dt}$$

Geometric Controller of a quadrotor

$$R_B^W = [x_B^W \ y_B^W \ z_B^W]$$

$$R_d^W = [x_d^W \ y_d^W \ z_d^W]$$



Steps for designing a geometric controller

- Input: desired position $P_d^w = [x_d, y_d, z_d]$ and desired yaw angle ψ_d
 - Output: propeller speed $\omega_i, i = 1, 2, 3, 4$
 - Compute $R_d^w = [x_d^w, y_d^w, z_d^w]$
1. Compute $\tilde{x}_d^w$ which is the projection of x_d^w on x-y plane (in the world frame)

$$\tilde{x}_d^w = [\cos(\psi_d), \sin(\psi_d), 0]^T$$
 3. compute z_d^w using equation (7.13) in lecture notes
1. Compute R_d^w using the equation right before 7.13 lecture notes and after 7.12
 2. Compute f_z^B and τ^B using equation (7.11) and 7.12
 3. Compute $\bar{F}$ from F in equation 6.9 by ignoring the first two rows
4.
$$\begin{bmatrix} \omega_1 |\omega_1| \\ \omega_2 |\omega_2| \\ \omega_3 |\omega_3| \\ \omega_4 |\omega_4| \end{bmatrix} = \bar{F}^{-1} \begin{bmatrix} f_z^B \\ \tau^B \end{bmatrix}$$

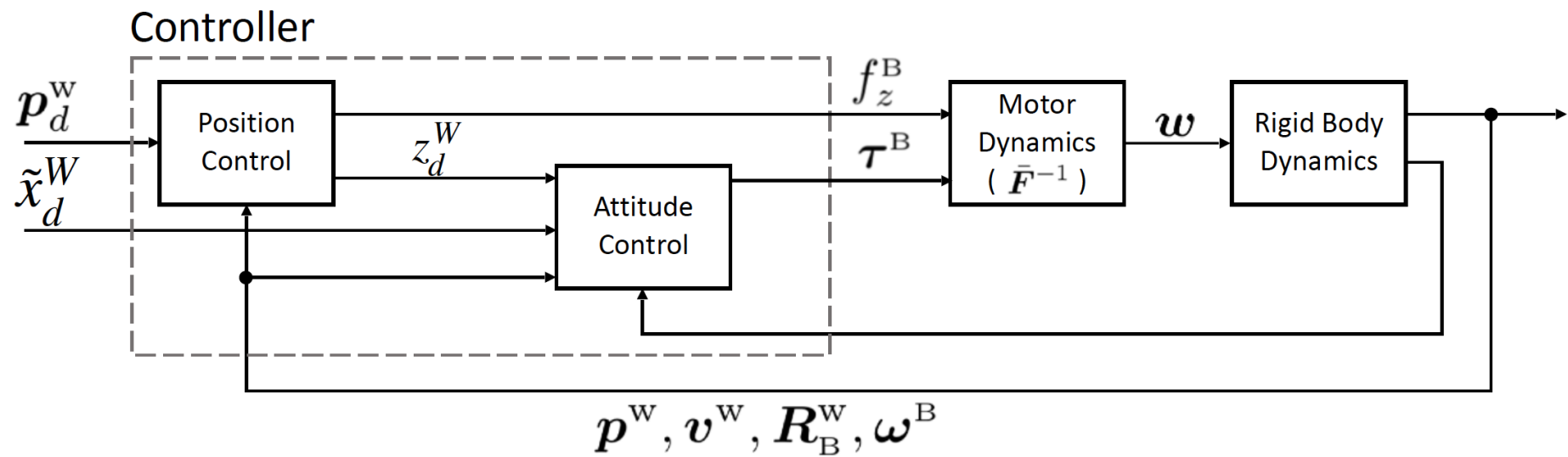


Figure 7.2: The geometric control loops for position and attitude control.

7.2.1 Tracking Errors and Control Design

In Lecture 6 we have seen that the quadrotor dynamics satisfy:

$$\begin{bmatrix} m\ddot{\mathbf{p}}^{\text{w}} \\ \mathcal{J}\dot{\boldsymbol{\omega}}^{\text{B}} \end{bmatrix} = \begin{bmatrix} -mge_3 \\ -\boldsymbol{\omega}^{\text{B}} \times \mathcal{J}\boldsymbol{\omega}^{\text{B}} \end{bmatrix} + \begin{bmatrix} \mathbf{R}_{\text{B}}^{\text{w}} & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_3 \end{bmatrix} \mathbf{F}\mathbf{w} \quad (7.1)$$

Noting that the first two rows of $\mathbf{F}$ are always zero in our quadrotor model, we rewrite eq. (6.9) in Lecture 6 as follows:

$$\begin{bmatrix} f_z^{\text{B}} \\ \boldsymbol{\tau}^{\text{B}} \end{bmatrix} = \bar{\mathbf{F}}\mathbf{w} \quad (7.2)$$

where $\bar{\mathbf{F}}$ is now a 4×4 invertible matrix. Since $\bar{\mathbf{F}}$ is constant, we can always move back and forth from $\mathbf{w}$ to $[f_z^{\text{B}} ; \boldsymbol{\tau}^{\text{B}}]$ and for this reason in the following we will directly think about $[f_z^{\text{B}} ; \boldsymbol{\tau}^{\text{B}}]$ as our control inputs and write (7.1) as:

$$\begin{bmatrix} m\ddot{\mathbf{p}}^{\text{w}} \\ \mathcal{J}\dot{\boldsymbol{\omega}}^{\text{B}} \end{bmatrix} = \begin{bmatrix} -mge_3 \\ -\boldsymbol{\omega}^{\text{B}} \times \mathcal{J}\boldsymbol{\omega}^{\text{B}} \end{bmatrix} + \begin{bmatrix} \mathbf{z}_{\text{B}}^{\text{w}} & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_3 \end{bmatrix} \begin{bmatrix} f_z^{\text{B}} \\ \boldsymbol{\tau}^{\text{B}} \end{bmatrix} \quad (7.3)$$

7.2.2 Tracking Errors

One might think about the geometric controller as a PD (Proportional Derivative) controller, but with metrics that respect the Lie group structure of the rotations. The errors are defined as follows:

$$\mathbf{e}_p = \mathbf{p}^w - \mathbf{p}_d^w \quad (\text{position error})$$

$$\mathbf{e}_v = \mathbf{v}^w - \mathbf{v}_d^w \quad (\text{linear velocity error})$$

$$\mathbf{e}_R = \frac{1}{2} [(\mathbf{R}_d^w)^\top \mathbf{R}_B^w - (\mathbf{R}_B^w)^\top \mathbf{R}_d^w]^\vee \quad (\text{rotation error})$$

$$\mathbf{e}_\omega = \boldsymbol{\omega}^B - (\mathbf{R}_B^w)^\top \mathbf{R}_d^w \boldsymbol{\omega}_d \quad (\text{angular velocity error})$$

7.2.3 Controller

We can now introduce the geometric controller. For given desired state $\mathbf{p}_d^w$, $\tilde{\mathbf{x}}_d^w$ (and the corresponding tracking errors $\mathbf{e}_p, \mathbf{e}_v, \mathbf{e}_R, \mathbf{e}_\omega$), and some positive constants k_p, k_v, k_R, k_ω , the control inputs $f_z^B, \boldsymbol{\tau}^B$ are chosen as follows:

$$f_z^B = (-k_p \mathbf{e}_p - k_v \mathbf{e}_v + mg \mathbf{e}_3 + m \ddot{\mathbf{p}}_d^w) \cdot \mathbf{R}_B^w \mathbf{e}_3, \quad (7.11)$$

$$\boldsymbol{\tau}^B = -k_R \mathbf{e}_R - k_\omega \mathbf{e}_\omega + \boldsymbol{\omega}^B \times \mathcal{J} \boldsymbol{\omega}^B - \mathcal{J}([\boldsymbol{\omega}^B]_\times (\mathbf{R}_B^w)^\top \mathbf{R}_d^w \boldsymbol{\omega}_d - (\mathbf{R}_B^w)^\top \mathbf{R}_d^w \dot{\boldsymbol{\omega}}_d), \quad (7.12)$$

where the desired attitude $\mathbf{R}_d^w$ is computed as $\mathbf{R}_d^w = [\frac{(\mathbf{z}_d^w \times \tilde{\mathbf{x}}_d^w) \times \mathbf{z}_d^w}{\|(\mathbf{z}_d^w \times \tilde{\mathbf{x}}_d^w) \times \mathbf{z}_d^w\|}, \frac{\mathbf{z}_d^w \times \tilde{\mathbf{x}}_d^w}{\|\mathbf{z}_d^w \times \tilde{\mathbf{x}}_d^w\|}, \mathbf{z}_d^w] \in \text{SO}(3)$, and:

$$\mathbf{z}_d^w = \mathbf{R}_d^w \mathbf{e}_3 = \frac{-k_p \mathbf{e}_p - k_v \mathbf{e}_v + mg \mathbf{e}_3 + m \ddot{\mathbf{p}}_d^w}{\| -k_p \mathbf{e}_p - k_v \mathbf{e}_v + mg \mathbf{e}_3 + m \ddot{\mathbf{p}}_d^w \|}, \quad (7.13)$$

Here, we assume that the denominator of (7.13) is non-zero,

$$\| -k_p \mathbf{e}_p - k_v \mathbf{e}_v + mg \mathbf{e}_3 + m \ddot{\mathbf{p}}_d^w \| \neq 0, \quad (7.14)$$

and that the desired trajectory satisfies

$$\| -mg \mathbf{e}_3 + m \ddot{\mathbf{p}}_d^w \| < B \quad (7.15)$$

for a given positive constant B .



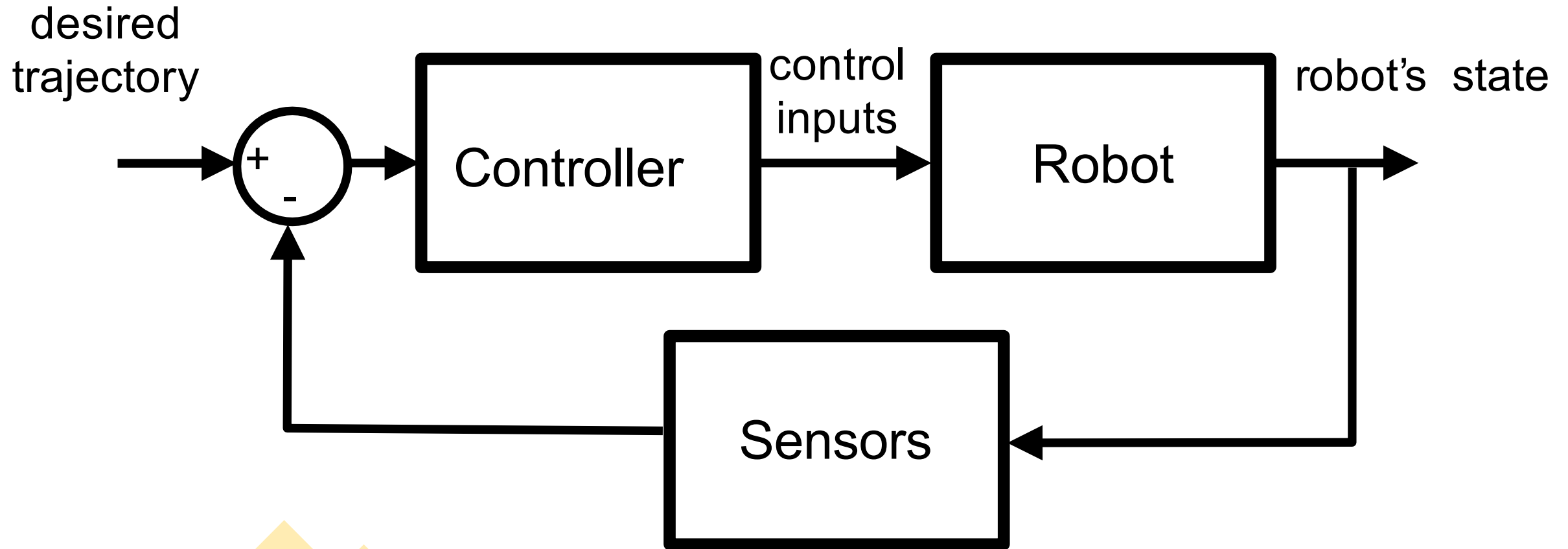
Computer Vision for Autonomous Vehicles

Trajectory Optimization

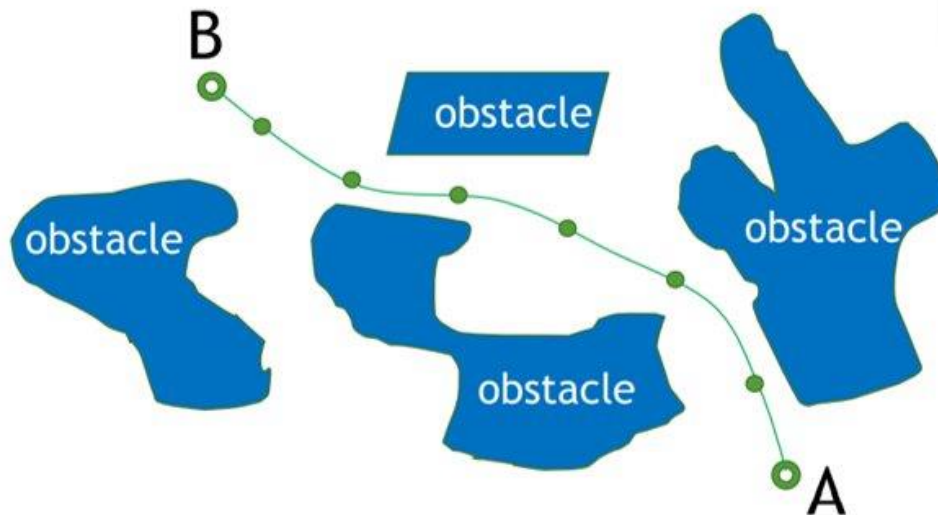
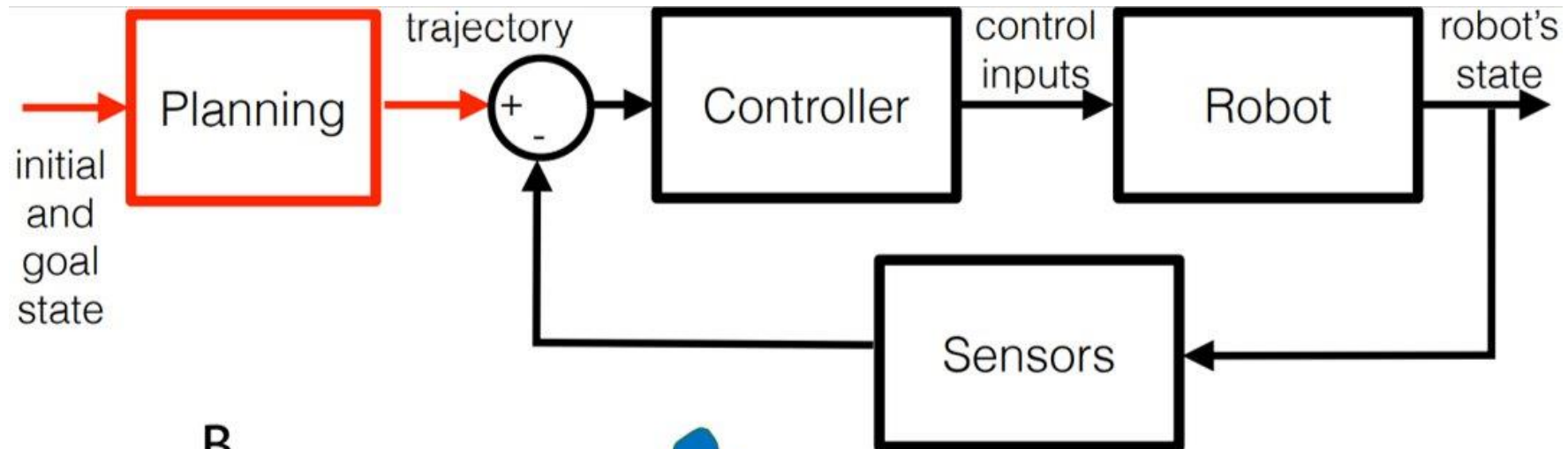
*Instructor: Golnaz Habibi, PhD
University of Oklahoma
Fall 2025*



Planning Vs. Control



Planning Vs control



Trajectory Planning vs. Path Planning

- Path vs. trajectory:
 - A path usually contains a sequence of waypoints, without time label or information about velocity or higher order of derivatives.
 - A trajectory defines the path the robot should navigate as a function of time.
- Path planning:
 - “slow” systems
 - Many approaches: graph-based algorithms (PRM, A* and variants), sampling-based methods (RRT, RRT* and variants)
- Trajectory planning:
 - Key for high-speed maneuvers on drones

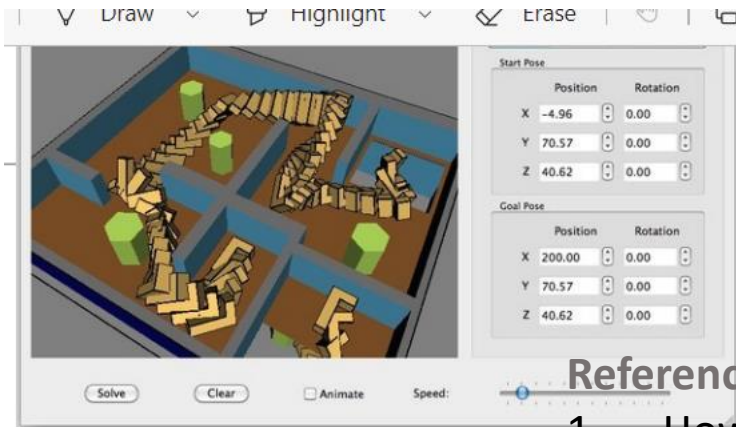
Motion Planning – Open Source Libraries

- Open Motion Planning Library (OMPL)

- <http://ompl.kavrakilab.org/>

- Motion Strategy Library (MSL)

- <http://msl.cs.uiuc.edu/msl/>

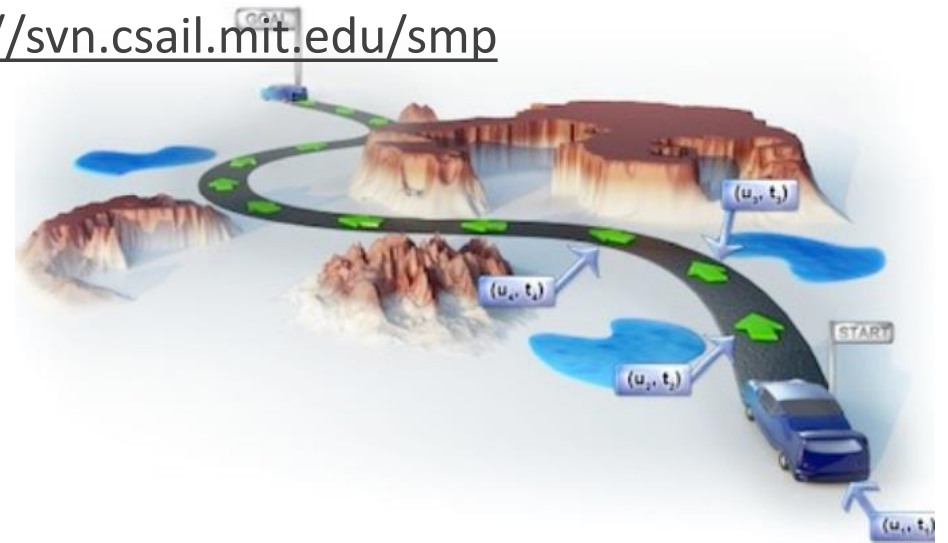


- RRT* Library

- <https://svn.csail.mit.edu/rrtstar/>

- Sampling Based Planning Library

- <https://svn.csail.mit.edu/smp>



References:

1. Howie Choset, Kevin Lynch et al, "Principles of Robot Motion" MIT press, 2005.
2. Steven Lavalle, "Planning Algorithms", Cambridge University Press, 2006.

Motion/Path Planning Approaches

COMBINATORIAL PLANNING

- (resolution) complete
 - They find the solution if exists, otherwise they report failure
- No need for collision checking, since is explicitly defined
- Expensive computation, impractical for higher dimensional systems
- Exploding number of cells
- The output path is usually jagged and needs smoothing

SAMPLING BASED PLANNING

- Probabilistic Complete
 - The probability of finding a solution if it exists tends to one, otherwise it may run forever.
- More practical
- Narrow passage challenge
- The output path is usually jagged and needs smoothing

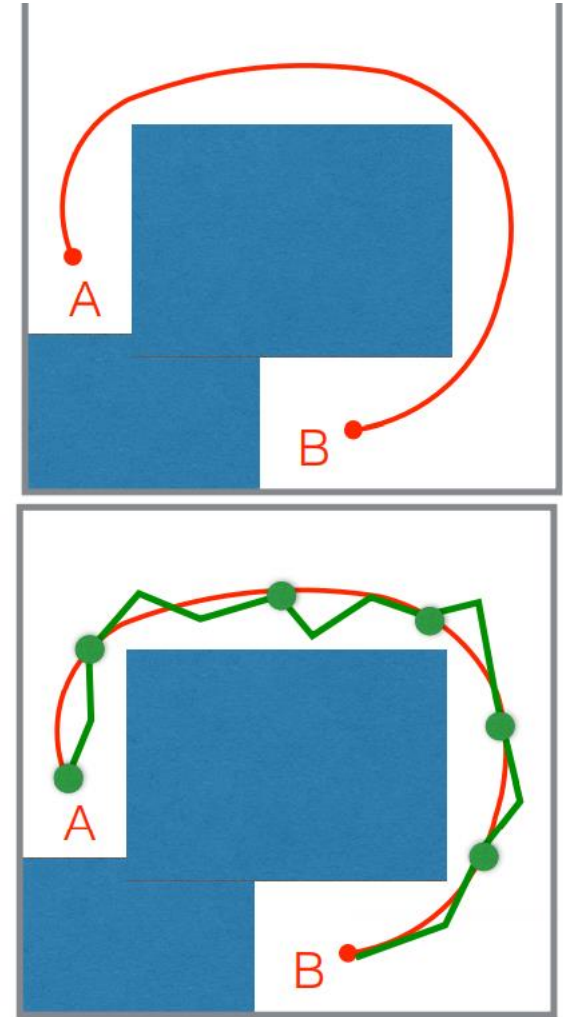
Trajectory Planning: Two Approaches

- **Direct trajectory planning:**

- plan the path and optimize trajectory simultaneously
- Typically slow for online planning and fast dynamics (difficult to account for obstacles)
- Fast heuristics: e.g., motion primitives

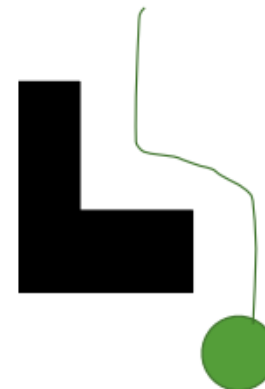
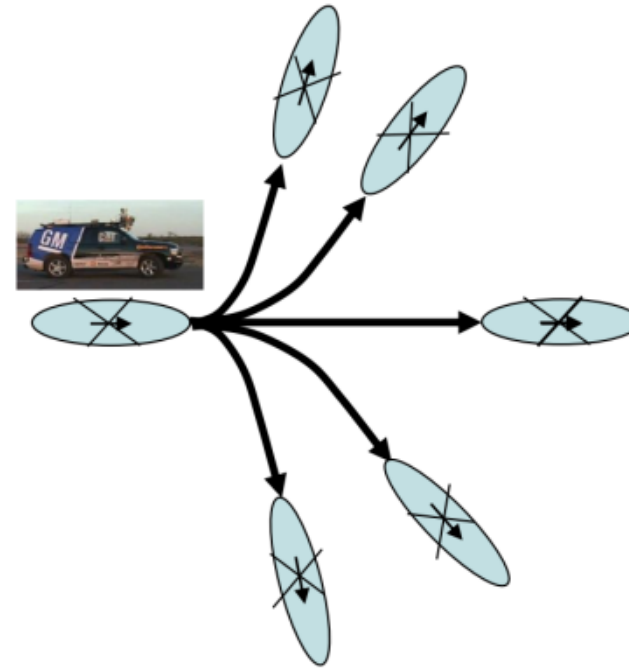
- **Decoupled trajectory planning:**

- first path planning then trajectory generation
- Fast computation (only partially accounts for obstacles)
- Polynomial trajectory optimization



Motion primitives

- **Input:**
 - A library of motion primitives: each motion primitive is generated by forward simulation of the system for a given control sequence
 - The current robot state
- **Output:** a choice of motion primitive that minimizes a cost function while avoiding obstacles.
- The final trajectory is usually not smooth
- More complex instantiations:
https://www.cs.cmu.edu/~maxim/files/tutorials/robschooltutorial_oct10.pdf



Number	Description
1	Straight
2	Climb
3	Takeoff (no throttle)
4	Gentle left
5	Gentle right
6	Left jog
7	Right jog

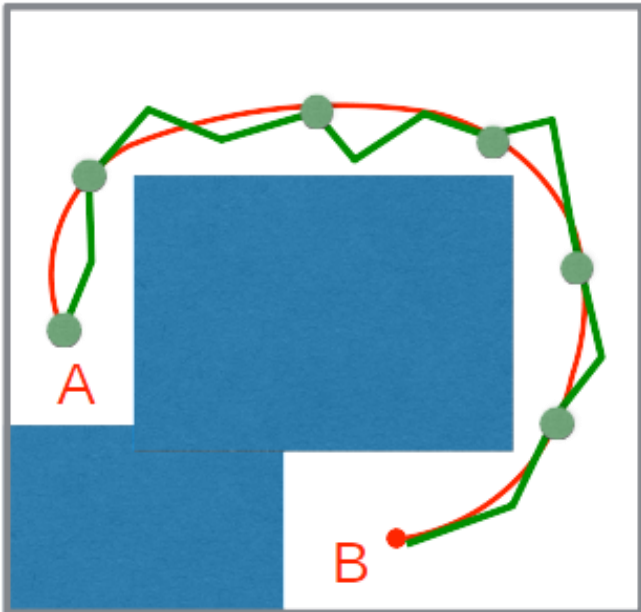
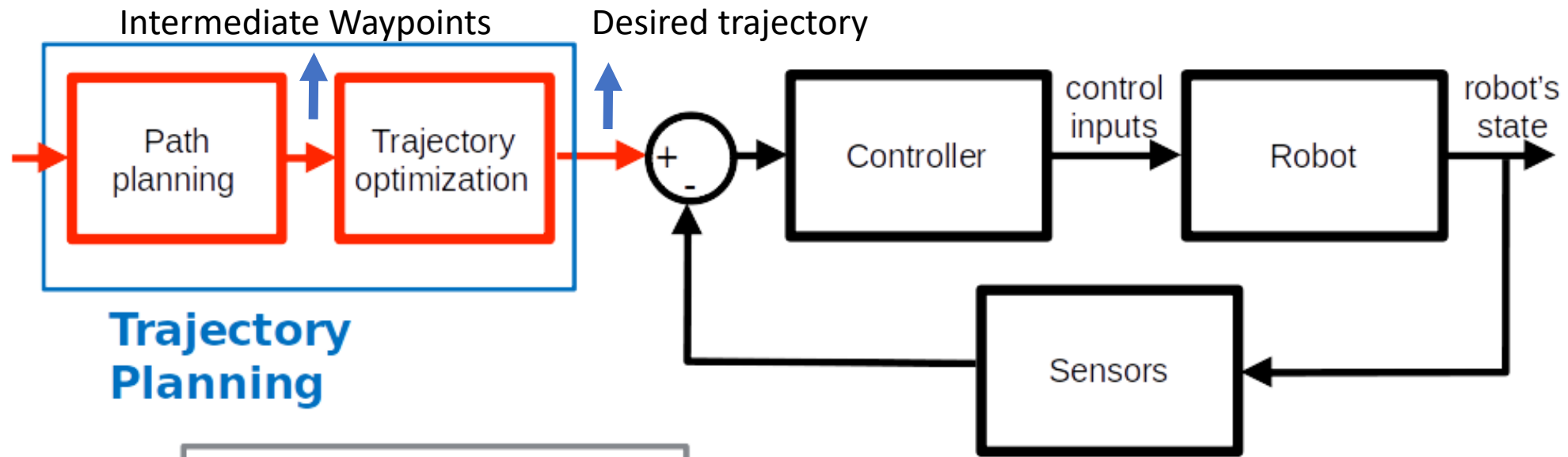
Output: 1->1>4->1->1->6

Motion primitives

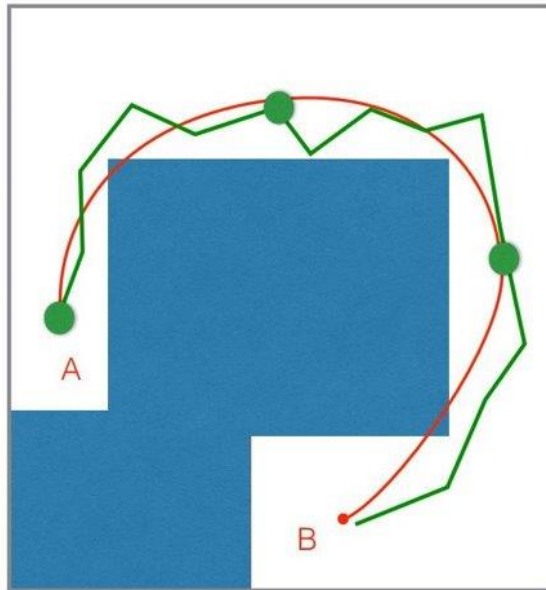


[Motion Primitives-based Path Planning for Fast and Agile Exploration using Aerial Robots - YouTube](#)

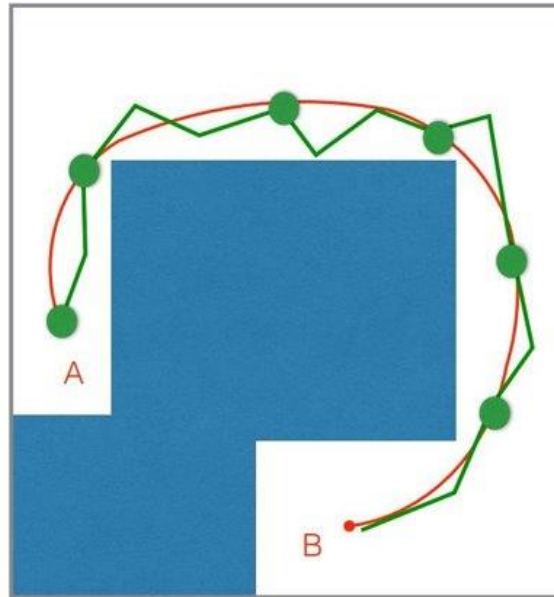
Decoupled Trajectory Planning



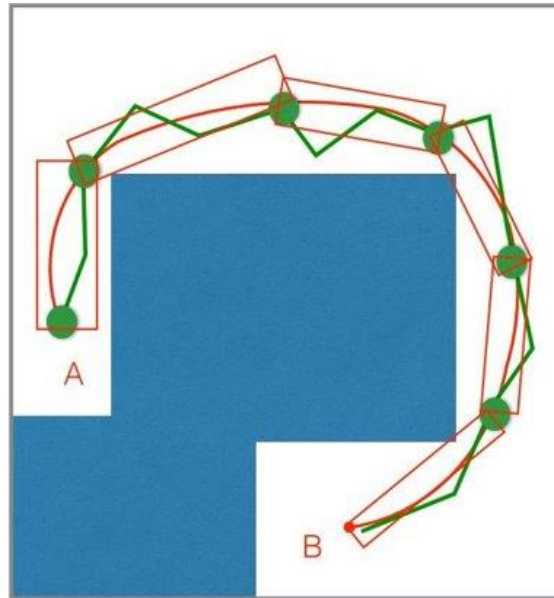
Collision Avoidance



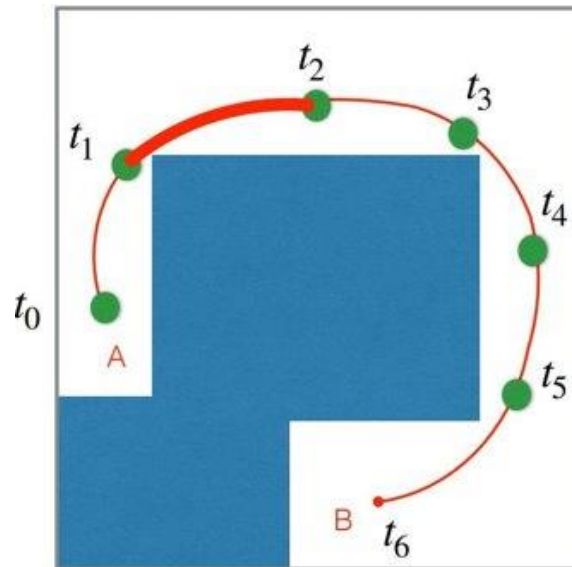
Collision Avoidance



Collision Avoidance



Time Allocation



Optimal Control Basics

$$\begin{aligned} & \min_{\mathbf{x}(t), \mathbf{u}(t)} && J(\mathbf{u}(t), \mathbf{x}(t)) \\ \text{subject to} &&& \dot{\mathbf{x}}(t) = f(\mathbf{x}(t), \mathbf{u}(t)) \\ &&& \mathbf{x}(0) = \bar{\mathbf{x}}_0 \\ &&& \mathbf{x}(T) = \bar{\mathbf{x}}_T \\ &&& \mathbf{u}_{min} \leq \mathbf{u}(t) \leq \mathbf{u}_{max} \\ &&& \mathbf{x}_{min} \leq \mathbf{x}(t) \leq \mathbf{x}_{max} \end{aligned}$$

Differential flatness

Our initial objective is to generate a smooth trajectory to allow the drone to fly from an initial to a final point. In lectures 6-7, we have shown the drone dynamics are differentially flat, which means there is a flat output $\boldsymbol{\sigma}(t) = [x, y, z, \psi]^\top$ such that the state and the input can be defined as smooth functions of this flat output and its derivatives, i.e., $\boldsymbol{x}(t) = h_x(\boldsymbol{\sigma}, \dot{\boldsymbol{\sigma}}, \ddot{\boldsymbol{\sigma}}, \dots)$ and $\boldsymbol{u} = h_u(\boldsymbol{\sigma}, \dot{\boldsymbol{\sigma}}, \ddot{\boldsymbol{\sigma}}, \dots)$.

Therefore, for differentially flat systems, we can rewrite the objective in (9.1) as:

$$J(\boldsymbol{u}(t), \boldsymbol{x}(t)) = J(h_u(\boldsymbol{\sigma}, \dot{\boldsymbol{\sigma}}, \ddot{\boldsymbol{\sigma}}, \dots), h_x(\boldsymbol{\sigma}, \dot{\boldsymbol{\sigma}}, \ddot{\boldsymbol{\sigma}}, \dots)) = J(\boldsymbol{\sigma}, \dot{\boldsymbol{\sigma}}, \ddot{\boldsymbol{\sigma}}, \dots) \quad (9.2)$$

Trajectory optimization in drone

- Drone property: differential flatness
- Minimum snap trajectory : we minimize the fourth derivatives

Minimum snap trajectory optimization

Cost functional. In lectures 6-7, we have seen that the drone inputs can be written in terms of the 4th derivative of the position of the drone (x, y, z) and the 2nd derivative of the drone yaw angle ψ . Therefore, to obtain a smooth trajectory (i.e., one with low input usage), we can minimize the 4th derivative of the positions (or, equivalently, the 4th derivative of the first three entry of our flat output $\boldsymbol{\sigma}(t) = [x, y, z, \psi]^T$) and the 2nd derivative of the yaw angle (or, equivalently, the 2nd derivative of the last entry of $\boldsymbol{\sigma}$):

$$J(\boldsymbol{\sigma}, \dot{\boldsymbol{\sigma}}, \ddot{\boldsymbol{\sigma}}, \dots) = \int_0^T (x^{(4)}(t))^2 dt + \int_0^T y^{(4)}(t)^2 dt + \int_0^T z^{(4)}(t)^2 dt + \int_0^T \psi^{(2)}(t)^2 dt \quad (9.3)$$

The previous expression represents the functional we want to minimize to obtain a smooth trajectory.

Polynomial Method

Therefore, the single-segment trajectory optimization problem for a flat polynomial output $x(t) = P(t)$ becomes:

$$\begin{aligned} & \underset{P(t)}{\text{minimize}} && \int_0^T (P^{(r)}(t))^2 dt \\ & \text{subject to} && P(0) = x_0 \\ & && \vdots \\ & && P^{(q)}(0) = x_0^{(q)} \\ & && P(T) = x_T \\ & && \vdots \\ & && P^{(q)}(T) = x_T^{(q)} \end{aligned} \tag{9.5}$$

Depending on the derivative r being minimized, the problem obtains different trajectories:

- $r = 1$ minimum velocity trajectory (shortest distance);
- $r = 2$ minimum acceleration trajectory;
- $r = 3$ minimum jerk trajectory;
- $r = 4$ minimum snap trajectory.

QP- based trajectory optimization

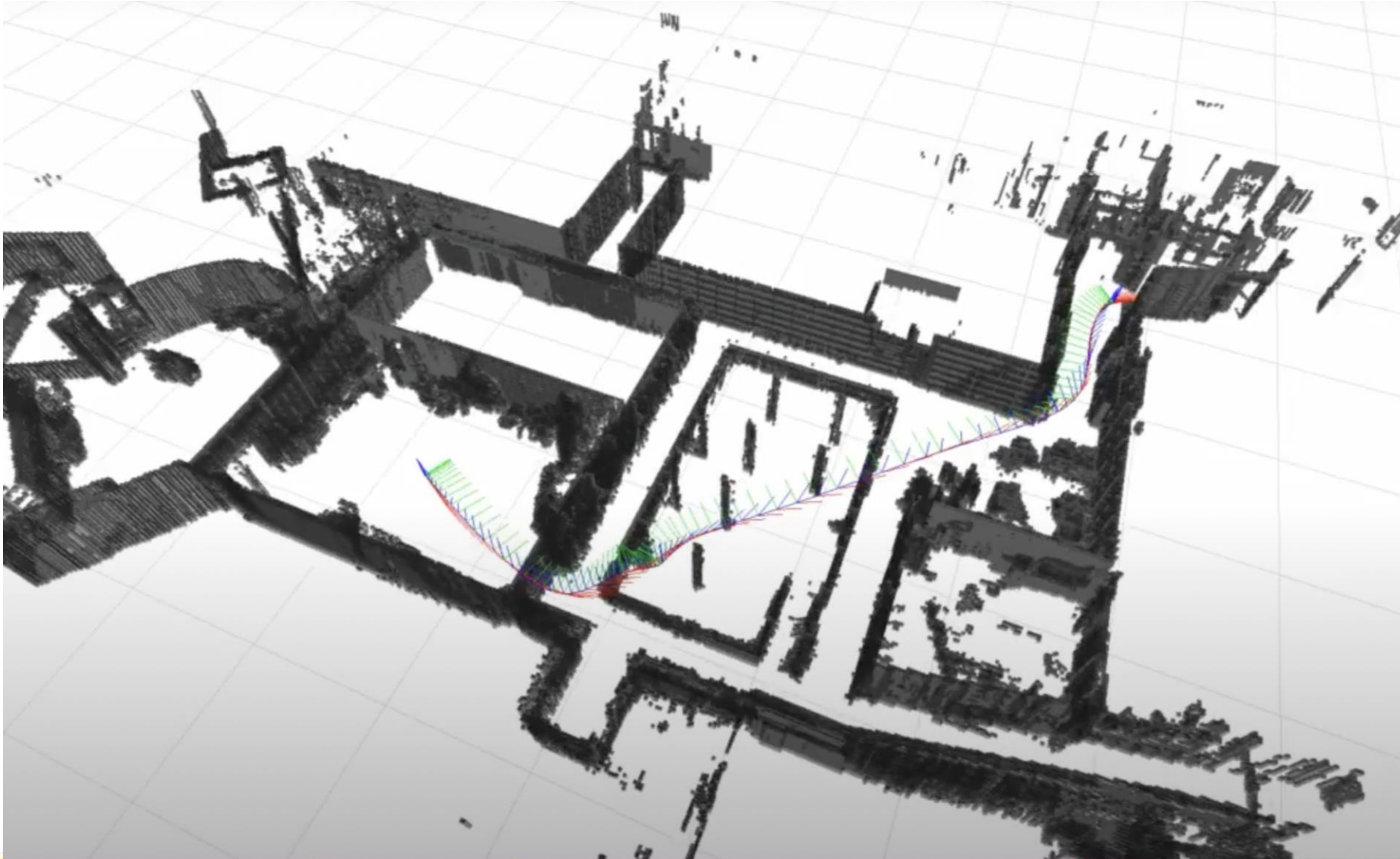
Assumption:

The order of the trajectory is **fixed**

Our trajectory optimization in this class is based on the following reference by Nick Roy's group at MIT:

*Charles Richter, Adam Bry, and Nicholas Roy, **Polynomial Trajectory Planning for Aggressive Quadrotor Flight in Dense Indoor Environments**, ISRR 2013*

Polynomial based Trajectory Optimization -ISRR 2013



Trajectory Optimization with waypoints

$$P(t) = \begin{cases} P_1(t), & t_0 \leq t \leq t_1 \\ P_2(t), & t_1 \leq t \leq t_2 \\ P_3(t), & t_2 \leq t \leq t_3 \end{cases} \quad (9.12)$$

Then the cost functional becomes:

$$J = \int_{t_0}^{t_1} \left(P_1^{(r)}(t) \right)^2 dt + \int_{t_1}^{t_2} \left(P_2^{(r)}(t) \right)^2 dt + \int_{t_2}^{t_3} \left(P_3^{(r)}(t) \right)^2 dt \quad (9.13)$$

Trajectory Optimization with waypoints

Motivation:

In many cases, we cannot fit only one long trajectory that passes all the waypoints created by a planner.

Solution:

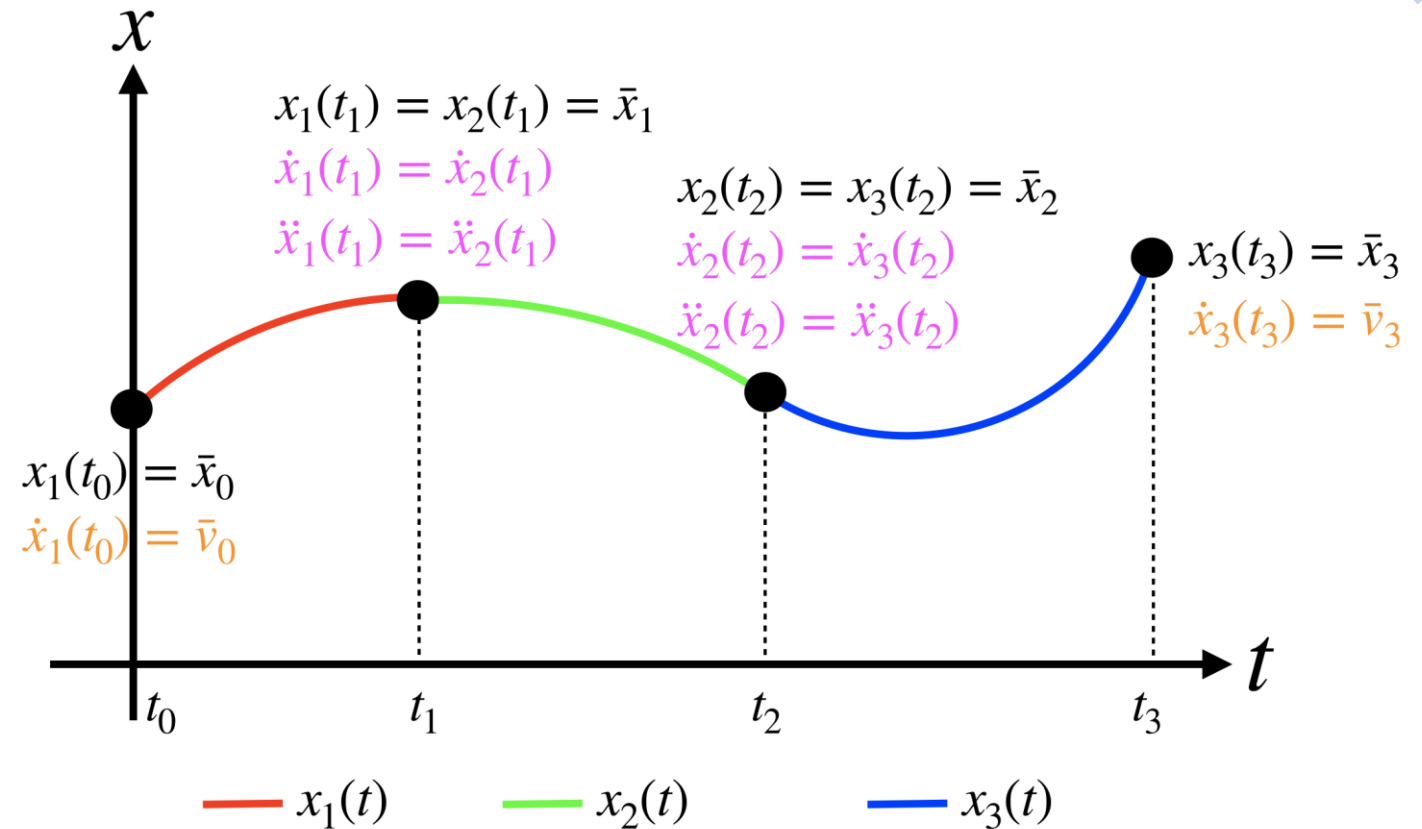
Consider one function (trajectory) between two waypoints

Challenge:

It requires preserving the continuity

Solution: Add the continuity constraints

- Continuous derivatives
- Continuous path



Trajectory Optimization with waypoints

$$P(t) = \begin{cases} P_1(t), & t_0 \leq t \leq t_1 \\ P_2(t), & t_1 \leq t \leq t_2 \\ P_3(t), & t_2 \leq t \leq t_3 \end{cases} \quad (9.12)$$

Then the cost functional becomes:

$$J = \int_{t_0}^{t_1} \left(P_1^{(r)}(t) \right)^2 dt + \int_{t_1}^{t_2} \left(P_2^{(r)}(t) \right)^2 dt + \int_{t_2}^{t_3} \left(P_3^{(r)}(t) \right)^2 dt \quad (9.13)$$

Unconstrained QP

To remove some constraints on the derivatives, one solution is to add the derivatives as decision variables of the trajectory optimization problem

Next Lecture

Computer vision
2-view Geometry